

Documentación Técnica de Arquitectura — Sistema FCMENU (Migración Python)

Proyecto: Migración del ERP de Facturación y Cuentas Corrientes **FCMENU** (Visual Basic 6.0 / MS-Access) a una aplicación Python moderna (PyQt6 / SQLAlchemy / PostgreSQL). **Alcance de este documento:** el árbol `migration/` de `c:\fcmenu` — la totalidad del código Python del proyecto — más su integración con el esquema PostgreSQL destino. **Audiencia:** ingeniería de plataforma, DBA, y cualquier desarrollador que deba extender, auditar o dar mantenimiento de largo plazo al sistema. **Fecha de corte del análisis:** 2026-08-18. Todas las referencias a líneas de código, conteos de filas y nombres de archivo fueron verificadas contra el estado real del repositorio a esa fecha.

Índice Técnico

- Capítulo 1 — Arquitectura del Sistema e Infraestructura
 - 1.1 Contexto y alcance
 - 1.2 Estructura del proyecto (mapa de directorio)
 - 1.3 Patrones de diseño y decisiones arquitectónicas autónomas
 - 1.4 Selección y justificación de librerías de terceros
 - 1.5 Diagrama arquitectónico de bloques
 - 1.6 Flujo de datos end-to-end (secuencia: emisión de Factura)
- Capítulo 2 — Anatomía del Código y Diccionario de Rutinas
 - 2.1 Capa de utilidades transversales
 - 2.2 Capa de acceso a datos — `db.py`
 - 2.3 Capa de acceso a datos — `repository.py`
 - 2.4 Capa de servicios — `services.py`
 - 2.5 Integración AFIP — `afip.py`
 - 2.6 Generación documental — `pdf.py` / `referencias.py`
 - 2.7 Capa de presentación (PyQt6) — patrones y componentes base
 - 2.8 Catálogo de ventanas y diálogos
 - 2.9 Entry points y ETL
- Capítulo 3 — Capa de Persistencia y Base de Datos (PostgreSQL)
 - 3.1 Modelo de datos — inventario de entidades
 - 3.2 Diagrama Entidad-Relación (ERD)
 - 3.3 Estrategia de conexión y Connection Pooling
 - 3.4 Ciclo de vida de sesiones
 - 3.5 Transacciones y resiliencia (ACID)
- Capítulo 4 — Manual de Mantenimiento, Extensibilidad y DevOps
 - 4.1 Guía de modificación segura
 - 4.2 Gestión de esquema — Alembic
 - 4.3 Trazabilidad, logging y manejo de errores
 - 4.4 Framework de pruebas

Capítulo 1 — Arquitectura del Sistema e Infraestructura

1.1 Contexto y alcance

`c:\fcmenu` contiene dos sistemas superpuestos:

1. **El sistema legacy** (raíz del repositorio): un ERP de Facturación, Cuentas Corrientes y Stock escrito en **Visual Basic 6.0**, con ~150 formularios/módulos (`.frm/.frx/.bas/.cls`), persistido en **MS-Access** (`fcmenu.mdb`) vía ADODB/DAO. Es la fuente de verdad funcional y normativa: toda regla de negocio migrada debe poder citarse contra una línea concreta de este código (política explícita en `CLAUDE.md`, "Regla de Oro #2: Cero Suposiciones en Reglas de Negocio").
2. **El sistema destino, objeto de este documento**: el paquete Python `migration/`, que reimplementa el ERP con una arquitectura en capas (persistencia → repositorio → servicio → presentación) sobre **PostgreSQL, SQLAlchemy 2.0** y **PyQt6**.

La migración avanza **formulario por formulario**, no de una sola vez (estrategia por fases impuesta por el usuario). Esto se refleja directamente en la estructura del código: cada ventana Python documenta en su docstring el `.frm` de origen, y `models.py` documenta en cada clase el criterio de auditoría de esquema usado (DAO 32-bit contra el `.mdb` real, no inferencia).

Alcance no cubierto por este documento: la lógica interna de los formularios VB6 no migrados aún, y el contenido de `fcmenu.mdb` como tal (más allá de su rol como fuente de esquema).

1.2 Estructura del proyecto (mapa de directorio)

```
c:\fcmenu\
|
|— CLAUDE.md                # Contrato de trabajo del proyecto: mapeo tecnológico
|                           # VB6→Python, reglas de oro de migración, flujo de trabajo.
|                           # Configuración raíz de Alembic (script_location, logging).
|— alembic.ini              # Dependencias de runtime (ver §1.4).
|— requirements.txt         # + pytest, alembic, pypdf (sólo para tests).
|— requirements-dev.txt    # Plantilla de variables de entorno (DATABASE_URL, entorno
|— .env.example            AFIP).
|                           # Base SQLite de desarrollo local (fallback si no hay
|— fcmenu.sqlite3          DATABASE_URL).
|                           # Base MS-Access ORIGEN (legacy, fuente de auditoría de
|                           esquema).
|— migration/              # ===== PAQUETE PYTHON – objeto de este documento =====
|   |— models.py           # Capa ORM: 21 clases declarativas SQLAlchemy (ver Cap. 3).
|   |— db.py               # Motor de conexión, connection pool, factory de sesiones.
|   |— repository.py       # Patrón Repository: 1 clase de acceso a datos por entidad.
|   |— services.py         # Capa de servicios: lógica de negocio pura (sin Qt, sin SQL
|                           directo).
|   |— decimals.py         # parse_decimal()/format_decimal() – E/S de moneda es-AR.
|   |— fechas.py           # Formato de fecha larga/corta en español (sin locale del
|                           SO).
|   |— provincias.py       # Catálogo estático de provincias argentinas (código →
|                           nombre).
|   |— referencias.py      # Generador de PDF con la matriz legacy de códigos de
|                           sección.
|   |— afip.py             # Cliente WSAA/WSFEv1 nativo (SOAP) – Facturación
|                           Electrónica AFIP.
|   |— pdf.py              # Generación de PDF (Factura/Recibo/Listados) con ReportLab.
|   |— examples.py         # Scripts de ejemplo de uso del RepositoryFactory (no
|                           productivo).
|   |— alembic/            # Migraciones de esquema versionadas (ver Cap. 4.2).
|   |   |— env.py          # Entry point de Alembic: liga
|                           target_metadata=Base.metadata.
```

```

| | | | script.py.mako # Plantilla para nuevas revisiones.
| | | | versions/ # 8 revisiones (esquema inicial + 7 incrementales, ver
$3.1).
| | | | etl/ # Extracción/carga única (migración de datos, no runtime).
| | | | | export_access_to_csv.ps1 # PowerShell: vuelca fcmenu.mdb → CSV vía DAO 32-bit.
| | | | | load_csv_to_postgres.py # Python: carga los CSV a PostgreSQL vía SQLAlchemy Core.
| | | | | data/*.csv # 16 volcados (uno por tabla migrada).
| | | | ui/ # Capa de presentación PyQt6 (ver §2.7-2.8).
| | | | | theme.py # Hoja de estilo global (QSS) + paleta corporativa.
| | | | | widgets.py # Controles reutilizables (QLineEdit numéricos, grillas,
foco).
| | | | | validators.py # cuit_valido()/email_valido() – validación de formato
puro.
| | | | | decimals.py # Re-export de formato Decimal para uso exclusivo de la
capa UI.
| | | | | iconos.py # Registro central de íconos (evita rutas hardcodeadas
repetidas).
| | | | | estado_conexion.py # Sondeo de conectividad a Internet (para el cartel de
AFIP).
| | | | | articulo_codigo.py # Motor de composición de Artículo.COD2 por segmentos.
| | | | | factura_renglon.py # Motor de cálculo de un renglón de Factura (DetFact.frm).
| | | | | detalle_grid.py # QAbstractTableModel genérico para grillas de detalle.
| | | | | stock_renglones_grid.py # Grilla especializada de movimientos de stock.
| | | | | main_menu_window.py # Ventana MDI principal (reemplaza FCMENU.frm) – 918
líneas.
| | | | | facturador_window.py # Emisor de Factura A/B (reemplaza EmiFact.frm) – 690
líneas.
| | | | | recibo_window.py # Emisor de Recibo (reemplaza EmiRec.frm) – 768 líneas.
| | | | | ctacte_window.py # Consulta de Cuenta Corriente (CtaCte.frm) – 371 líneas.
| | | | | tablas_window.py # ABM de tablas paramétricas (Tablas.frm) – 676 líneas.
| | | | | listados_window.py # Motor de listados/reportes (Listados.frm) – 559 líneas.
| | | | | cliente_detalle_dialog.py # ABM de Cliente (Abmclte.frm) – 531 líneas.
| | | | | articulo_detalle_dialog.py # ABM de Artículo (AbmArt.frm) – 417 líneas.
| | | | | arreglo_subdiario_window.py # Corrección manual de Subdiario IVA – 409 líneas.
| | | | | pago_dialog.py # Grilla de aplicación de pagos del Recibo – 325 líneas.
| | | | | stock_movimiento_window.py # Alta de movimiento de stock – 279 líneas.
| | | | | parametros_window.py # Edición de parámetros globales (Paramet.frm) – 285
líneas.
| | | | | arreglo_ctacte_window.py # Corrección manual de Cuenta Corriente – 257 líneas.
| | | | | ventas_articulo_window.py # Estadística de ventas por artículo – 248 líneas.
| | | | | mod_precios_window.py # Modificación masiva de precios (ModPrec.frm) – 243
líneas.
| | | | | cheque_detalle_dialog.py # ABM de Cheque de terceros – 233 líneas.
| | | | | cliente_busqueda_window.py # Búsqueda de Clientes (BusClte.frm) – 218 líneas.
| | | | | cheques_consulta_window.py # Consulta de cartera de cheques (VerCheq.frm) – 193
líneas.
| | | | | cobranzas_zona_window.py # Cobranzas pendientes por zona/vendedor – 188 líneas.
| | | | | cotizacion_window.py # Alta de cotización diaria de dólar – 175 líneas.
| | | | | ventas_seccion_window.py # Estadística de ventas por sección – 169 líneas.
| | | | | total_diario_detalle_dialog.py # Detalle de un día de Totales – 200 líneas.
| | | | | dtos_cliente_dialog.py # Descuentos por Cliente/Sección (DtosexClte.frm) – 203
líneas.
| | | | | stock_consulta_window.py # Consulta de stock (Stock.frm) – 150 líneas.
| | | | | totales_diarios_window.py # Listado de Totales diarios (TotFact.frm) – 153 líneas.
| | | | | facturas_emitidas_window.py # Consulta de facturas emitidas (VerFact.frm) – 147
líneas.
| | | | | articulo_busqueda_window.py # Búsqueda de Artículos – 200 líneas.
| | | | | despacho_selector_dialog.py # Selector de lote/despacho con stock – 123 líneas.
| | | | | factura_detalle_dialog.py # Detalle de un renglón cargado – 131 líneas.

```

```

├─ factura_emitida_detalle_dialog.py # Detalle de factura ya emitida – 129 líneas.
├─ acerca_de_dialog.py             # Diálogo "Acerca de" – 120 líneas.
├─ nota_cliente_dialog.py          # Notas libres por Cliente (NotaClte.frm) – 116 líneas.
├─ cheques_cliente_dialog.py       # Cheques en cartera de un Cliente – 117 líneas.
├─ nota_articulo_dialog.py         # Notas libres por Artículo (Notartic.frm) – 113 líneas.
├─ despachos_consulta_window.py    # Consulta de lotes/despachos – 111 líneas.
├─ procesando_dialog.py           # Modal de progreso indeterminado – 95 líneas.
├─ banco_búsqueda_dialog.py        # Selector de Banco/Sucursal – 84 líneas.
├─ movstock_historial_dialog.py    # Historial de movimientos de un artículo – 71 líneas.
├─ pdf_preview_dialog.py           # Visor embebido de PDF generado – 69 líneas.
├─ total_mensual_dialog.py         # Resumen mensual de Totales – 63 líneas.
├─ saldos_clientes_dialog.py       # Listado de saldos de todos los clientes – 64 líneas.
├─ lote_detalle_dialog.py          # Detalle de un lote/despacho puntual – 59 líneas.
├─ main.py / main_*.py (24)       # Entry points standalone, uno por ventana (ver §2.9).
├─ __init__.py
├─ tests/                          # Suite pytest (ver Cap. 4.4).
│   ├── conftest.py                # Fixture `db`: SQLite en memoria, aislado por test.
│   └── test_repository.py         # 693 líneas – regresiones de bugs reales de la capa
Repository.
│   ├── test_services.py           # 2545 líneas – la suite más grande: reglas de negocio.
│   ├── test_afip.py               # QR AFIP, generación/firma de TRA.
│   └── test_pdf.py                # Contenido real del PDF generado (vía pypdf), no sólo
existencia.
│   ├── test_factura_renglon.py    # Motor de renglón de Factura.
│   ├── test_articulo_codigo.py    # Motor de composición de código de Artículo.
│   ├── test_decimals.py / test_fechas.py / test_provincias.py / test_ui_validators.py
│   └── __init__.py

```

Nota de exclusión deliberada. El árbol de arriba omite los ~150 archivos legacy VB6 (`.frm/.frx/.bas/.cls`) en la raíz de `c:\fcmenu`: son la *fuentes* de la migración, no parte del sistema Python documentado aquí. La correspondencia formulario-legacy → módulo-Python está anotada en cada entrada de arriba y se detalla en la [tabla de catálogo de ventanas](#) (§2.8).

1.3 Patrones de diseño y decisiones arquitectónicas autónomas

La migración no siguió un framework de aplicación (ej. Django, FastAPI+algo) — es una aplicación de escritorio PyQt6 con una arquitectura en capas construida a medida. Los patrones aplicados, y por qué:

Patrón	Dónde	Justificación
Repository	<code>repository.py</code> , una clase por entidad (<code>ClienteRepository</code> , <code>ArticuloRepository</code> , ...) heredando de <code>BaseRepository[T]</code> (Generic CRUD)	Aísla el código de consulta SQLAlchemy del resto del sistema. Cada método de negocio en <code>services.py</code> recibe datos ya tipados (<code>Cliente</code> , <code>list[Ctascte]</code> , ...) sin conocer la sesión ORM subyacente. Facilita testear servicios contra SQLite en memoria (ver <code>tests/conftest.py</code>) sin acoplarse al dialecto PostgreSQL.
Service Layer	<code>services.py</code> (3.309 líneas, ~20 clases de servicio)	Concentra <i>toda</i> la lógica de negocio (cálculo de IVA, cascada de descuentos, imputación de pagos, emisión de comprobantes) fuera de la UI y fuera del acceso a datos. Ninguna clase de <code>ui/</code> calcula un total o decide una regla de negocio: sólo arma el <code>dict</code> /dataclass de entrada y llama al servicio. Esto es lo que permite que <code>test_services.py</code> (2.545 líneas) cubra toda la lógica crítica sin levantar Qt .
Factory	<code>RepositoryFactory</code> (<code>repository.py:1330</code>)	Punto único de construcción de repositorios atados a una <code>Session</code> : <code>repos = RepositoryFactory(db); repos.cliente()</code> . Evita que cada servicio reconstruya

Patrón	Dónde	Justificación
		<code>ClienteRepository(db)</code> manualmente y centraliza qué repositorios existen.
DTO / Value Object immutable	<code>@dataclass</code> en <code>services.py</code> (<code>TotalFactura</code> , <code>ResultadoEmision</code> , <code>ExtractoAnual</code> , <code>CobranzasZona</code> , <code>ResultadoRecibo</code> , ...) y en <code>pdf.py</code> (<code>DatosFacturaPDF</code> , <code>DatosReciboPDF</code>)	Los servicios nunca devuelven <code>dicts</code> sueltos ni tuplas posicionales para resultados compuestos: devuelven dataclasses tipadas. Esto le da a la UI autocompletado/verificación de tipos al consumir el resultado, y documenta el contrato de salida sin necesidad de leer el cuerpo del método.
Strategy	<code>NumeracionYCAEProvider</code> (ABC, <code>afip.py:80</code>) con dos implementaciones: <code>AfipWSFEv1Stub</code> (sin red, CAE simulado) y <code>AfipWSFEv1Cliente</code> (SOAP real contra AFIP)	<code>FacturadorWindow</code> recibe el proveedor por inyección de dependencia (<code>afip: Optional[NumeracionYCAEProvider] = None</code> , <code>facturador_window.py:93</code>) — permite testear la ventana y correr en desarrollo sin certificado AFIP real, y cambiar a producción con una sola variable de entorno (<code>FCMENU_AFIP_ENTORNO</code> , ver <code>.env.example</code>).
Template Method / control de foco compartido	<code>EnterAsTabFilter</code> (<code>widgets.py:278</code>)	Un único <code>QObject</code> instalado como <code>eventFilter</code> en cada ventana reproduce el comportamiento "Enter avanza como Tab" del VB6 original (pedido explícito de usabilidad en <code>CLAUDE.md §3</code>) sin reescribir el manejo de foco en cada formulario.
Fachada de tema (Facade)	<code>aplicar_tema(app)</code> (<code>theme.py:233</code>)	Una sola llamada en cada <code>main()</code> centraliza paleta de colores, hoja de estilo QSS y configuración global de <code>QComboBox</code> (ver <code>_instalar_limite_combos</code> , que parchea <code>QComboBox.__init__</code> para fijar un límite de filas visibles en todos los combos de la app sin tocar cada instancia).
Composite ligero de validación	<code>_caracter_permitido()</code> en la jerarquía <code>_NumericLineEditBase</code> → <code>MontoLineEdit</code> / <code>EnteroLineEdit</code> / <code>AlfanumericoLineEdit</code> (<code>widgets.py:132-268</code>)	Reemplaza los controles OCX de terceros (ComponentOne, DBI-Tech) mencionados en <code>CLAUDE.md</code> por subclases de <code>QLineEdit</code> con validación de tecla por tecla, sin dependencias externas de UI.

Decisión explícita de NO usar un ORM "activo" (Active Record): las clases de `models.py` son puramente declarativas (sin métodos de negocio, sin `save()`/`delete()` propios) — toda operación pasa por `repository.py`. Esto mantiene la regla de la capa de servicios de no depender de detalles de persistencia y es lo que permite que `services.py` no importe nunca `sqlalchemy.orm.Session` más que como tipo de parámetro.

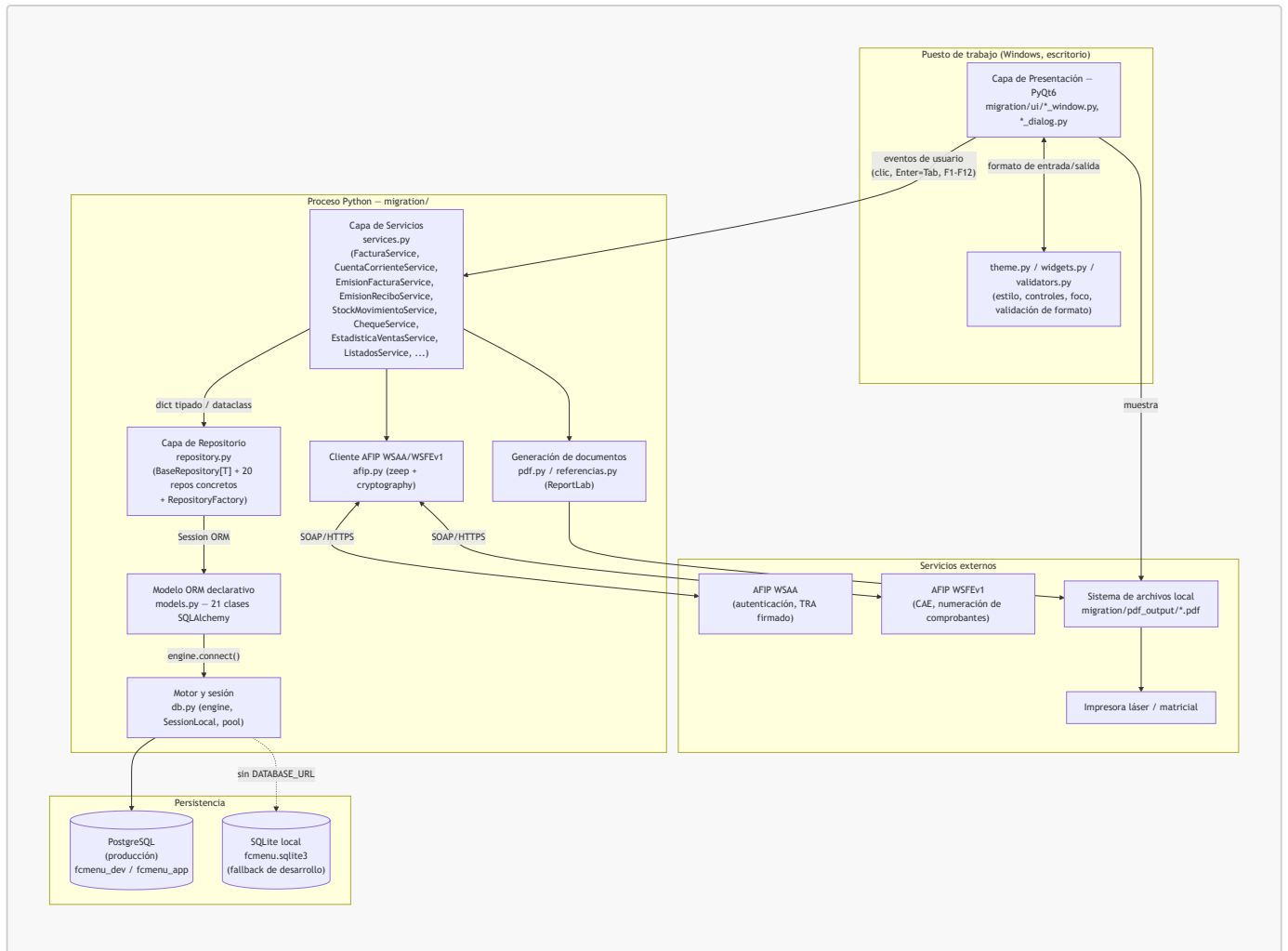
Decisión explícita de NO envolver excepciones de negocio en una jerarquía propia amplia: la mayoría de las violaciones de regla de negocio se señalizan con `ValueError` simple y un mensaje descriptivo en español (ver `services.py:1373`, `:1378`, `:1742`, etc.) que la capa UI muestra tal cual en un `QMessageBox.critical`. La única excepción son los dos errores de dominio de `factura_renglon.py` (`SeccionInexistenteError`, `SeccionSinUnidadFacturacionError`, ambas subclases de `ValueError`), que sí ameritan tipo propio porque la UI necesita distinguirlas de otros errores de validación para dar un mensaje más específico.

1.4 Selección y justificación de librerías de terceros

Librería	Versión mínima	Rol	Justificación registrada
SQLAlchemy	≥2.0	ORM + Core, motor de conexión, Alembic autogenerate	Estándar de facto en Python para acceso relacional tipado; API 2.0 (<code>Mapped[]</code> , <code>mapped_column</code>) da <code>type hints</code> nativos en el modelo, alineado con el requisito del proyecto de tipar todo.

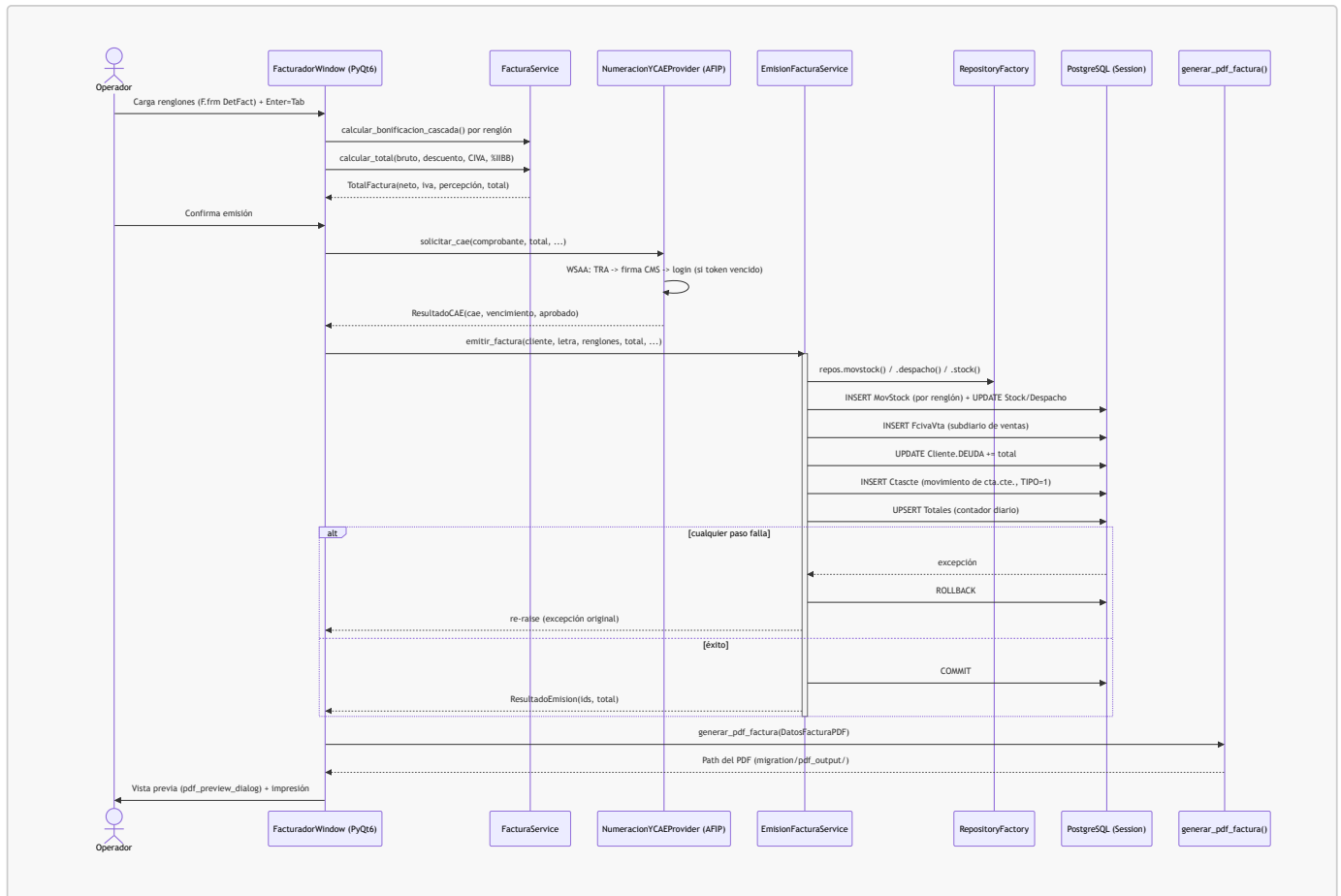
Librería	Versión mínima	Rol	Justificación registrada
psycopg2-binary	≥2.9	Driver DBAPI de PostgreSQL	Driver síncrono maduro; variante -binary evita depender de libpq /headers de compilación en la máquina de desarrollo Windows.
PyQt6	≥6.7	Framework de interfaz gráfica de escritorio	Reemplazo directo del runtime de formularios VB6; elegido (frente a PySide6) por decisión de licencia/soporte del proyecto, documentado en CLAUDE.md §3 ("PyQt6 (o PySide6)"). QTableView/QAbstractTableModel reemplazan los grids OCX de terceros (ComponentOne FlexGrid, CodeJock, DBI-Tech) que exigía el legacy.
cryptography	≥42.0	Firma CMS/PKCS7 del TRA (Ticket de Requerimiento de Acceso) para WSAA	AFIP exige el TRA firmado en CMS binario DER; cryptography provee primitivas de firma sin depender de OpenSSL como binario externo.
zeep	≥4.2	Cliente SOAP para WSAA y WSFEv1	AFIP expone Facturación Electrónica como dos servicios SOAP/WSDL estándar. Decisión explícita, registrada en CLAUDE.md y confirmada 2026-08-16: no se usa pyafipws (el wrapper histórico de la comunidad) — su paquete de PyPI falla al instalar en este entorno (pip rechaza el .tar.gz por un symlink que escapa del directorio de extracción, protección real de pip) y no es necesario: WSAA/WSFEv1 son sólo dos servicios SOAP, cubiertos con zeep + cryptography sin el wrapper. El circuito completo (TRA→firma CMS→SOAP) fue validado en vivo contra el WSDL de Homologación real de AFIP (FEDummy respondió; una autenticación de prueba con certificado autofirmado fue rechazada con el error real esperado, confirmando el circuito de punta a punta).
reportlab	≥4.0	Generación de PDF (Factura, Recibo, Listados)	El QR de AFIP se dibuja nativamente con reportlab.graphics.barcode.qr — decisión explícita de no sumar una librería de QR aparte, ya cubierto por una dependencia que el proyecto ya necesitaba para el PDF.
pytest (dev)	≥8.0	Framework de pruebas	Estándar de la comunidad Python; fixtures (tests/conftest.py) e id paramétrico usados extensivamente en test_services.py .
alembic (dev)	≥1.13	Versionado de esquema	Integración nativa con SQLAlchemy 2.0 declarative; soporta --autogenerate contra Base.metadata .
pypdf (dev)	≥5.0	Sólo en tests/test_pdf.py , para leer de vuelta el PDF generado y verificar su contenido textual (CAE, vencimiento, total)	Verificar el PDF por contenido real, no sólo por existencia del archivo — decisión de calidad de pruebas explícita en el docstring del test.

1.5 Diagrama arquitectónico de bloques



1.6 Flujo de datos end-to-end (secuencia: emisión de Factura)

Se documenta el circuito más crítico del sistema — `FacturadorWindow._on_emitir()` (`ui/facturador_window.py:498`) — porque atraviesa las cinco capas y es el de mayor riesgo transaccional (afecta stock, cuenta corriente y subdiario de IVA en una sola operación atómica).



Puntos de decisión de diseño visibles en este flujo:

- El CAE se solicita **antes** de persistir el comprobante — si AFIP rechaza, no se toca la base. Si AFIP aprueba pero la persistencia falla, el comprobante queda con CAE "huérfano" (riesgo operativo conocido, mismo comportamiento que el legacy; ver recomendación en §4.1).
- Toda la escritura multi-tabla de `EmisionFacturaService.emitir_factura()` ocurre dentro de un único `try/except` con `commit()` al final y `rollback()` + `raise` ante cualquier excepción (`services.py:1385-1451`) — ver detalle en §3.5.

Capítulo 2 — Anatomía del Código y Diccionario de Rutinas

Criterio de cobertura. Este capítulo documenta con ficha técnica completa (Propósito, Gatillo, Contrato, Excepciones) toda función y método **crítico** — entendido como: motor de cálculo de negocio, punto de entrada transaccional, o utilidad transversal reusada por múltiples capas. Los ~40 archivos de ventana/diálogo de `ui/` comparten 2-3 patrones estructurales idénticos (documentados una sola vez en §2.7) y se catalogan en tabla en §2.8 para evitar 40 fichas técnicamente redundantes entre sí; sus métodos privados de construcción de layout (`_armar_*`, `_construir_ui`) no tienen valor de mantenimiento adicional al código mismo y se omiten de la ficha, salvo cuando contienen lógica de negocio (caso de `facturador_window.py`, documentado en detalle).

2.1 Capa de utilidades transversales

Estas utilidades viven en `migration/` (no en `migration/ui/`) explícitamente porque tanto `services.py` como la capa `ui/` las necesitan, y `services.py` tiene prohibido depender de `ui/` (regla de capas, ver §1.3).

`parse_decimal(texto: str) -> Decimal` — `decimals.py:17`

Propósito	Convierte texto tipeado por el operador (convención es-AR: coma decimal, punto de miles) a <code>Decimal</code> , sin pasar nunca por <code>float</code> .
Gatillo	Cada vez que un campo de entrada numérico (<code>MontoLineEdit</code> y afines) necesita el valor tipado del usuario como número exacto — al confirmar un renglón, al recalcular un total, al validar un formulario.
Contrato	<code>texto: str</code> → <code>Decimal</code> . Texto vacío o no parseable → <code>Decimal("0")</code> (nunca levanta excepción al llamador).
Excepciones	Captura internamente <code>decimal.InvalidOperation</code> y degrada a <code>Decimal("0")</code> — decisión deliberada de "nunca romper la UI por un typo", documentada en el propio docstring.

`format_decimal(valor: Decimal, decimales: int = 2) -> str` — `decimals.py:31`

Propósito	Formatea un <code>Decimal</code> en convención es-AR ("1.234,56") — la inversa simétrica de <code>parse_decimal()</code> .
Gatillo	Cualquier punto donde un importe se muestra en pantalla o se arma como texto que puede volver a pasar por <code>parse_decimal()</code> (ej. al poblar una celda editable de grilla).
Contrato	<code>(valor: Decimal, decimales: int = 2)</code> → <code>str</code> en formato "1.234,56".
Excepciones	No captura excepciones; asume <code>valor</code> convertible a <code>Decimal</code> .
Nota de mantenimiento crítica	El docstring documenta un bug real de producción (2026-08-15, cliente con Debe \$681.240,23): formatear con <code>f"{valor:,.2f}"</code> directo (formato US) y releer con <code>parse_decimal()</code> corrompe silenciosamente cualquier importe ≥ 1.000, porque la coma se reinterpreta como separador decimal. Regla de oro para cualquier extensión futura: nunca usar <code>f"{valor:,.2f}"</code> directamente en código que toque montos — siempre pasar por <code>format_decimal()</code>.

`formatear_fecha_larga(fecha: date) -> str` / `formatear_fecha_corta(fecha: date) -> str` — `fechas.py:23` / `fechas.py:34`

Propósito	Formato de fecha en español ("lunes 15 ago 2026" / "Dom 16 Ago 2026") sin depender de que el locale <code>es-AR</code> esté instalado en el SO donde corre la app.
Gatillo	Grilla de Pagos del Recibo (<code>formatear_fecha_larga</code>); cabecera de <code>MainMenuWindow</code> (<code>formatear_fecha_corta</code> , formato más compacto).

Contrato	<code>date</code> → <code>str</code> . Tablas fijas <code>_DIAS_ES/_MESES_ABREV_ES/_DIAS_ABREV_ES</code> indexadas por <code>fecha.weekday()/fecha.month</code> .
Excepciones	Ninguna manejada explícitamente; <code>fecha</code> debe ser un <code>datetime.date</code> válido.

`nombre_provincia(codigo: str | None) -> str` — `provincias.py:49`

Propósito	Traduce el código de provincia (2 caracteres, convención Access del legacy) al nombre completo, para reportes y pantallas.
Gatillo	Listados (<code>ListadosService</code>), armado de PDF de factura/recibo (necesitan el nombre completo, no el código).
Contrato	<code>str None</code> → <code>str</code> . Código no encontrado o <code>None</code> → cadena vacía o el propio código (verificar <code>provincias.py</code> ante cualquier extensión del catálogo).

2.2 Capa de acceso a datos — `db.py`

`init_db()` -> `None` — `db.py:38`

Propósito	Crea todas las tablas declaradas en <code>Base.metadata</code> si no existen (<code>Base.metadata.create_all</code>).
Gatillo	Bootstrap de un entorno nuevo sin pasar por Alembic (ej. entorno de desarrollo rápido contra SQLite). No se usa contra PostgreSQL en flujos donde Alembic ya gestiona el esquema — ver advertencia en §4.2.
Contrato	Sin parámetros. Efecto secundario únicamente (DDL).
Excepciones	Propaga cualquier excepción de SQLAlchemy/DBAPI sin capturar.

`get_session()` -> `Session` — `db.py:42`

Propósito	Factory de sesiones ORM: <code>SessionLocal()</code> (autoflush=False, autocommit=False, future=True).
Gatillo	Cada ventana de <code>ui/</code> la invoca una vez, típicamente en su <code>__init__</code> , y mantiene la sesión abierta durante toda la vida de la ventana.
Contrato	Sin parámetros → <code>Session</code> nueva, no compartida. El llamador es responsable de cerrarla (no hay context manager ni <code>try/finally</code> en el llamador por defecto en la mayoría de las ventanas).
Excepciones	No maneja excepciones; construcción directa del <code>sessionmaker</code> .
Nota de mantenimiento crítica	Este patrón ("una sesión por ventana, nunca cerrada hasta que el usuario cierra la ventana a mano") es la causa raíz documentada de un incidente real: con el pool por defecto de SQLAlchemy (5+10=15 conexiones), abrir ~18 pantallas sin cerrarlas agotaba el pool y la ventana N°19 se colgaba sin error visible. Mitigado subiendo <code>pool_size=30</code> , <code>max_overflow=20</code> (ver §3.3), no cambiando el patrón de ciclo de vida — cualquier extensión que agregue más ventanas simultáneas debe revisar este límite.

2.3 Capa de acceso a datos — `repository.py`

`repository.py` (1.463 líneas) implementa **20 repositorios concretos** + `RepositoryFactory`, todos heredando de `BaseRepository[T]`. Se documenta el genérico completo y una selección de métodos concretos con lógica no trivial (traducción de reglas de negocio del `.mdb` real, no CRUD plano).

`class BaseRepository(Generic[T])` — `repository.py:37`

Método	Contrato	Gatillo	Notas
<code>create(obj_in: dict[str, Any]) -> T</code>	Instancia el modelo con <code>**obj_in, add() + commit() + refresh()</code>	Alta de cualquier entidad	Commit inmediato — no participa de una transacción más amplia salvo que el llamador la administre aparte (ver <code>EmissionFacturaService</code> , que no usa <code>create()</code> sino <code>db.add()</code> directo para controlar el commit único, §3.5)
<code>read(obj_id: int) -> Optional[T]</code>	<code>SELECT ... WHERE id = obj_id</code> (PK sintética, no la clave de negocio)	Cualquier consulta por PK interna	—
<code>read_all(skip=0, limit=100) -> List[T]</code>	<code>SELECT</code> paginado	Listados genéricos	Límite por defecto 100 — cualquier pantalla que necesite más debe pasar <code>limit</code> explícito
<code>update(obj_id, obj_in: dict) -> Optional[T]</code>	<code>setattr</code> por cada clave + <code>commit() + refresh()</code>	Edición genérica	Devuelve <code>None</code> si <code>obj_id</code> no existe, sin excepción
<code>delete(obj_id: int) -> bool</code>	<code>db.delete() + commit()</code>	Baja física (no hay soft-delete en el esquema)	Devuelve <code>False</code> si no existía

`ClienteRepository.proximo_codigo() -> int` — `repository.py:145`

Propósito	Calcula el próximo código de cliente disponible (<code>MAX(CODIGO)+1</code>), reproduciendo la asignación autoincremental manual que hacía el VB6 (la tabla no tiene identity nativo sobre <code>CODIGO</code> , que es clave de negocio, no la PK sintética <code>id</code>).
Gatillo	Alta de nuevo cliente desde <code>cliente_detalle_dialog.py</code> .
Contrato	Sin parámetros → <code>int</code> .

`ClienteRepository.proximo_correlativo(cliente: Cliente) -> int (staticmethod)` — `repository.py:163`

Propósito	Determina el siguiente correlativo de comprobante para un cliente, a partir de <code>CORR1/CORR2</code> del propio registro <code>Cliente</code> .
Gatillo	Emisión de comprobante que requiere numeración correlativa por cliente (no por punto de venta global).

`DtoxClteRepository.reemplazar_para_cliente(clte: int, filas: list[dict]) -> int` — `repository.py:221`

Propósito	Reemplaza <i>todas</i> las filas de descuento por sección de un cliente (borra + inserta), en vez de hacer upsert fila a fila — refleja cómo <code>DtosxClte.frm</code> graba: la grilla completa se resincroniza de una vez.
Gatillo	Confirmación del diálogo <code>dtos_cliente_dialog.py</code> .
Contrato	<code>(clte: int, filas: list[dict]) → int</code> (cantidad de filas insertadas).
Excepciones	Propaga excepciones de integridad de SQLAlchemy sin capturar; el llamador (servicio/UI) decide cómo mostrarlas.

`ArticuloRepository.by_cod1_cod2(cod1: str, cod2: str) -> Optional[Articulo]` — `repository.py:296`

Propósito	Búsqueda por clave de negocio compuesta (COD1 + COD2 , Sección+Código), la clave real del artículo en el .mdb (no la PK sintética id).
Nota de mantenimiento crítica	El docstring de ArticuloRepository (y el de Despacho , que replica el mismo patrón) documenta que COD1/COD2 en el .mdb real vienen con padding fijo de Access (longitud fija con espacios). Cualquier comparación debe considerar ese padding — confirmado empíricamente contra datos reales, no asumido.

StockRepository.criticos() -> **List[Stock]** — repository.py:379

Propósito	Artículos por debajo de stock mínimo.
Nota de mantenimiento crítica	tests/test_repository.py documenta un bug real ya corregido: la implementación original filtraba por EST1 (que el proceso de venta no actualiza) en vez de STUNID (el campo que sí refleja el stock real post-venta). Cualquier refactor de este método debe preservar el test de regresión asociado.

CtascteRepository.deuda_cliente(clte: int) -> **dict[str, Any]** — repository.py:629

Propósito	Calcula la deuda total de un cliente a partir de los movimientos de Ctasctes .
Nota de mantenimiento crítica	tests/test_repository.py documenta un bug real ya corregido: la versión original sumaba mal DEBE en vez de IMPTE/TIPO — es decir, no distinguía correctamente débito de crédito por tipo de movimiento. Es el método de mayor sensibilidad contable del sistema; cualquier cambio requiere validación explícita del usuario (regla de oro del proyecto).

ParametroRepository.get_ivains() -> **Optional[Parametro]** — repository.py:559

Propósito	Recupera la fila de configuración con la alícuota de IVA Inscripto.
Nota de mantenimiento crítica	Bug real corregido: la implementación buscaba CLAVE='IVAINS' (valor que no existe en la tabla real) en vez de la fila única con CLAVE='1' . La tabla Parametro actúa como fila de configuración singleton, no como catálogo clave-valor genérico — cualquier extensión debe respetar ese diseño de una única fila activa.

CotizacionRepository — repository.py:1275, métodos **_parse_fecha/_format_fecha/ultima/by_fecha/guardar**

Propósito	Persistencia de la cotización diaria del dólar.
Nota de mantenimiento crítica	Cotizacion.FECHA está modelado como String(10) (no Date) porque así está en el .mdb real (type=10, TEXT(10) , probablemente "dd/mm/yyyy") — es la única tabla del sistema con este quirk . En consecuencia ultima()/by_fecha() no pueden usar ORDER BY/WHERE FECHA = :fecha directo en SQL (el orden lexicográfico de texto no coincide con el orden cronológico); el repositorio resuelve esto parseando en Python (_parse_fecha) y comparando en memoria. Documentado explícitamente como caso especial en tests/test_repository.py .

DespachoRepository.resumen_lotes() -> **list[dict]** — repository.py:430

Propósito	Agrega el stock disponible por lote/despacho (NRODESP) para el selector de lote de DetFact.frm/despacho_selector_dialog.py .
------------------	--

Gatillo

Al elegir un artículo en el Facturador, si el artículo maneja lotes con stock (**STOCK** > 0) se ofrece la selección puntual en vez de descontar del stock general.

RepositoryFactory — repository.py:1330

Punto único de construcción; expone un método por entidad (**cliente()**, **articulo()**, **stock()**, **despacho()**, **movstock()**, **totales()**, **parametro()**, **ctascte()**, **imputacion()**, **fciva_vta()**, **fcestad1()**, **fctablas()**, **proveed()**, **banco()**, **cheque()**, **efectivo()**, **movimvs()**, **cotizacion()**, **notaclte()**, **dtoxclte()**, **notaarticulo()**), cada uno perezoso (crea el repositorio la primera vez que se pide, lo cachea en la instancia).

2.4 Capa de servicios — services.py

services.py (3.309 líneas) es el módulo de mayor densidad de reglas de negocio del sistema. **Todas** las fórmulas están citadas contra la línea exacta del **.frm** de origen en el docstring del método (política del proyecto, ver §1.1) — este documento reproduce esas citas para no duplicar información que pueda desincronizarse del código.

FacturaService — services.py:87

Cálculo puro de totales de factura (IVA, bonificaciones, percepciones). **No** persiste ni emite comprobantes — esa responsabilidad es de **EmissionFacturaService**.

Método	Ficha técnica
<code>alicuota_iva_inscripto() -> Decimal</code>	Propósito: lee la alícuota de IVA configurable desde Parametro.IVAINS (fila CLAVE='1'), fuente FCMENU.bas:333-343 . Gatillo: cada cálculo de total de factura. Contrato: sin parámetros → Decimal (fracción, ej. 0.21). Excepciones: ValueError explícito si Parametro no está configurado — replica que "el legacy directamente aborta el arranque en este caso".
<code>percepcion_iibb_habilitada() -> bool</code>	Propósito: replica ConPercep <> 1 de EmiFact.frm:1177 sobre Parametro.MCAIB . Contrato: MCAIB nulo → percepción deshabilitada (mismo default que FCMENU.bas:350-354).
<code>calcular_bonificacion_cascada(importe: Decimal, porcentajes: list[Decimal]) -> Decimal</code>	Propósito: aplica hasta 5 niveles de descuento en cascada (cada porcentaje sobre el saldo ya descontado, no sumados entre sí). Contrato: fórmula exacta — SumDtos += (Impte - SumDtos) * (Dto_i/100) , réplica literal de EmiFact.frm:1821-1827 . Gatillo: por cada renglón de factura, usando los 5 niveles de DtoxClte .
<code>letra_comprobante(civa_cliente: int, provincia: Optional[str]) -> str</code>	Propósito: determina Factura "A" o "B". Contrato: "A" si civa_cliente < 3 o (civa_cliente == 4 y provincia "V" tras .strip()); "B" en cualquier otro caso. Fórmula confirmada contra CabFact.frm:709-713 . Nota: el .strip() es una corrección deliberada del padding fijo de Access sobre Cliente.PCIA (viene como 'V ').
<code>calcular_total(bruto, descuento, civa_cliente, porcentaje_iibb=0) -> TotalFactura</code>	Propósito: arma el desglose completo. Contrato: neto_gravado = bruto - descuento ; IVA y percepción IIBB sólo se calculan si civa_cliente < 3 (gravado); redondeo a 2 decimales ROUND_HALF_UP en cada paso (_round2 , equivalente al Round() de VB6). Nota de negocio documentada: la rama de "Responsable No Inscripto" (ClteCIVA=2 , cálculo TotIVANI/TotSIVA) es código legacy muerto — categoría de IVA eliminada por AFIP en 2003 — y

Método	Ficha técnica
	se excluye deliberadamente del cálculo (decisión validada con el usuario).

CuentaCorrienteService — services.py:305

Método	Ficha técnica
saldo_cliente(clte: int) -> Decimal	Saldo actual de cuenta corriente de un cliente.
facturas_pendientes(clte: int) -> list	Facturas con saldo impago.
tiene_deuda_vieja(clte: int, hoy=None) -> bool	Determina si hay comprobantes vencidos hace más de X (regla configurable) — usado para bloquear nuevas ventas a crédito.
cobranzas_por_zona(zona: int, hoy=None) -> CobranzasZona	Agrega deuda pendiente de cobro agrupada por zona/vendedor, para el reporte de cobranzas.
imputar_pago(...) -> ImputacionResultado (services.py:454)	Propósito: aplica un pago (Recibo) contra uno o más comprobantes pendientes, reproduciendo la lógica FIFO de imputación del legacy (slots <code>IMPUT1..IMPUT6</code> de <code>Ctascte</code>). Gatillo: confirmación de un Recibo con pagos manuales imputados a facturas concretas (no a cuenta). Contrato: devuelve <code>ImputacionResultado</code> con <code>comprobante_id</code> y <code>slot_usado</code> — el llamador (<code>EmisionReciboService</code>) usa este resultado para decidir qué slot de <code>Ctascte</code> marcar como saldado. Esta es la lógica de mayor sensibilidad contable del sistema junto con <code>deuda_cliente()</code> .
extracto_anual(clte: int, anio: int) -> ExtractoAnual	Extracto de movimientos de cuenta corriente de un cliente para un año, con saldo inicial arrastrado (ver <code>CtascteRepository.saldo_inicial</code>).
resumen_cliente(cliente: Cliente) -> ResumenCliente	Resumen consolidado (saldo, deuda vieja, últimas facturas) para la cabecera de pantallas de cliente.
saldos_todos_clientes() -> list[dict]	Fuente del listado "Saldos de Cuenta Corriente" (<code>saldos_clientes_dialog.py</code>).

StockService / StockMovimientoService — services.py:688 / services.py:792

Método	Ficha técnica
StockService.articulos_criticos() / a_reponer()	Delegan en <code>StockRepository.criticos()</code> (ver nota de bug corregido en §2.3) para alimentar el panel de reposición.
StockService.registrar_movimiento(...) (services.py:719)	Alta de un movimiento de stock genérico (ajuste manual, no ligado a un comprobante).
StockMovimientoService.validar_seccion(cod_seccion: str) -> None (services.py:838)	Propósito: valida que la sección exista en <code>Fctabla1</code> y tenga unidad de facturación configurada antes de permitir cargar un movimiento. Excepciones: <code>ValueError("La Sección {cod} No Existe en la Tabla")</code> / <code>ValueError("La Sección '{cod}' no se vende por unidades")</code> .
StockMovimientoService.emitir_movimiento(...) -> ResultadoMovimientoStock (services.py:849)	Propósito: orquesta el alta completa de un movimiento de stock con sus renglones. Contrato:

Método	Ficha técnica
	valida forma de movimiento (<code>ValueError("Forma de movimiento inválida")</code>), exige N° de despacho si es importación, exige al menos un renglón, y verifica que no exista ya un movimiento con ese número (<code>ValueError("Ya existe un movimiento con ese NÚMERO")</code>) — replica las validaciones de guardia del formulario legacy antes de tocar la base.
<code>StockMovimientoService._grabar_renglon(...)</code> (<code>services.py:910</code>)	Inserta un renglón de <code>MovStock</code> y actualiza <code>Stock/Despacho</code> en consecuencia; es privado porque sólo tiene sentido invocado dentro de la transacción de <code>emitir_movimiento()</code> .

ClienteService / ArtículoService / TablaService — `services.py:988 / :1077 / :1191`

Capa fina de reglas de negocio de ABM (búsqueda, validación de baja) sobre `ClienteRepository/ArticuloRepository/FctablasRepository`.

Método	Ficha técnica
<code>ClienteService.puede_dar_de_baja(clte) -> tuple[bool, str]</code> (<code>services.py:1019</code>)	Contrato: (<code>permitido</code> , <code>motivo</code>) — verifica que el cliente no tenga saldo pendiente ni movimientos históricos antes de habilitar la baja física. El <code>str</code> explica al operador por qué no puede darse de baja cuando <code>permitido=False</code> .
<code>ArticuloService.calcular_precio_con_porcentaje(precio, porcentaje) -> Decimal</code> (<code>services.py:1124, staticmethod</code>)	Aplica un porcentaje de aumento/descuento a un precio — motor puro reutilizado por <code>candidatos_modificacion_precio()</code> .
<code>ArticuloService.candidatos_modificacion_precio(...) -> ...</code> (<code>services.py:1138</code>)	Filtra artículos por rango <code>Val(COD2)</code> (réplica de la función <code>Val()</code> de VB6, ver <code>_val_vb6</code>) para el modificador masivo de precios (<code>ModPrec.frm</code>).
<code>ArticuloService.aplicar_modificacion_precio(articulos, porcentaje) -> int</code> (<code>services.py:1175</code>)	Aplica el cambio a la lista completa en una sola operación; devuelve la cantidad de artículos afectados.
<code>TablaService.puede_dar_de_baja(ctab, cod) -> tuple[bool, str]</code> (<code>services.py:1219</code>)	Igual patrón que <code>ClienteService</code> , para filas de la tabla paramétrica genérica <code>Fctabla1</code> .

EmissionFacturaService — `services.py:1300`

El servicio transaccional más crítico del sistema — ver ficha completa de `emitir_factura()` ya detallada en §1.6 y el fragmento de código citado ahí (`services.py:1350-1458`). Métodos auxiliares:

Método	Ficha técnica
<code>_dias_vencimiento(cvta: Optional[int]) -> int</code> (<code>services.py:1461</code>)	Propósito: días de la Condición de Venta del cliente (<code>Fctabla1 CTAB='CV'</code> , <code>NUMSD3</code>), réplica de <code>Graba()</code> líneas 2274-2285. Contrato: 30 días si no se encuentra la condición de venta (mismo default del legacy).
<code>_grabar_renglon(...)</code> (<code>services.py:1472</code>)	Inserta <code>Fcestad1</code> (línea estadística) y <code>MovStock</code> por cada renglón de la factura, y descuenta <code>Stock/Despacho</code> según corresponda.

Método	Ficha técnica
<code>_bonificacion_renglon(clte, cod_seccion, importe) -> Decimal (services.py:1548)</code>	Resuelve los 5 niveles de <code>DtoxClte</code> para el cliente+sección del renglón y delega en <code>FacturaService.calcular_bonificacion_cascada()</code> .
<code>_upsert_totales(...)</code> (services.py:1560)	Actualiza (o crea, si es el primer comprobante del día) la fila <code>Totales</code> del día — contador agregado diario de facturación.

EmissionReciboService — services.py:1650

Método	Ficha técnica
<code>emitir_recibo(...) -> ResultadoRecibo</code> (services.py:1706)	Propósito: orquesta el alta de un Recibo completo: aplicación de pagos (<code>AplicacionPago</code> , vía <code>CuentaCorrienteService.imputar_pago</code>), cheques recibidos (<code>PagoCheque</code> → tabla <code>Cheques</code>), retenciones (<code>PagoRetencion</code> → tabla <code>MovimVS</code>) y efectivo (tabla <code>Efectivo</code>). Réplica de <code>EmiRec.frm Sub Graba()</code> . Excepciones: <code>ValueError("El Recibo no tiene ningún importe a cobrar.")</code> si la suma de todos los medios de pago es cero; validaciones adicionales de consistencia entre el total declarado y la suma de aplicaciones. Contrato transaccional: igual patrón <code>try/db.add()*N/commit() + except: rollback(); raise</code> que <code>EmissionFacturaService.emitir_factura()</code> .

ChequeService — services.py:1926

Método	Ficha técnica
<code>buscar(estado, fecha_desde, limite=50) -> list[Cheque]</code>	Búsqueda de cheques por estado ("En Cartera", "Rechazado", ...) desde una fecha.
<code>registrar_egreso(...)</code> (services.py:1971)	Propósito: mueve un cheque de "En Cartera" a un destino de egreso (depósito, endoso, etc.). Excepciones: <code>ValueError(f"No existe el cheque N° {nrocheq}")</code> , <code>ValueError(f"El cheque N° {nrocheq} no está En Cartera.")</code> , <code>ValueError("Destino de Egreso inválido.")</code> — tres guardas de negocio antes de cualquier escritura.
<code>eliminar(nrocheq: int) -> None</code> (services.py:2002)	Baja física de un cheque; misma guarda de existencia que <code>registrar_egreso</code> .

EstadisticaVentasService — services.py:2080

Motor de agregación para los reportes de ventas por sección/artículo (`ESTADIST.frm`/`VTAXART.frm`).

Método	Ficha técnica
<code>fin_de_mes_exclusivo(fecha: date) -> date</code> (staticmethod)	Calcula el primer día del mes siguiente — patrón usado en todo el sistema para rangos de fecha "hasta exclusive", evitando el error off-by-one de comparar contra el último día del mes.
<code>ventas_seccion_por_rango(cod_seccion, fecha_desde, fecha_hasta) -> VentasSeccionRango</code>	Total de ventas de una sección en un rango, con desglose de columnas configurable.
<code>ventas_articulo_agrupadas(...)</code> -> <code>VentasArticuloAgrupadas</code> (services.py:2197)	Agrupamiento jerárquico de ventas por artículo según <code>ConfigAgrupamiento</code> (nivel de agregación configurable por sección).

Método	Ficha técnica
<code>_agrupar(...)</code> / <code>_subtotales(...)</code> (privados)	Motor genérico de agrupamiento reusado por el método público de arriba.

FacturasEmitidasService / TotalesDiariosService — services.py:2302 / :2460

Servicios de sólo-lectura para reportes de facturación ya emitida y de totales diarios/mensuales (`VerFact.frm`/`TotFact.frm`). `TotalesDiariosService._dias_habiles(anio, mes, hasta_dia)` y `_division_segura(numerador, denominador)` son utilidades privadas para el cálculo de promedios diarios sin división por cero.

ArregloCtaCteService / ArregloSubdiarioService — services.py:2636 / :2737

Herramientas de **corrección manual** de un movimiento puntual de Cuenta Corriente / Subdiario de IVA — reemplazan `ZZGenDB.frm`/`Acttabla.frm`. Ambos siguen el mismo patrón: `buscar()` → `grabar()` (con `_resincronizar_deuda(clte)` posterior, que recalcula `Cliente.DEUDA` desde cero tras la corrección) → `borrar()`. **Uso restringido por diseño:** son la única vía del sistema para editar directamente un movimiento contable ya persistido — cualquier extensión de permisos sobre estas pantallas debe tratarse como cambio de control interno, no como feature de UI.

ListadosService — services.py:2937

Fachada de reportes (`Listados.frm`): `listado_clientes`, `lista_precios`, `subdiario_ventas`, `deuda_pendiente`, `estado_cuenta`, `saldos_cta_cte`, `subdiario_cobranzas`, `ingresos_brutos`, `subdiario_ventas_comisiones`, `percepciones_arba`, `comisiones_cobranzas`. Cada método arma la lista de filas tipadas que luego consume `pdf.py` (`generar_pdf_listado`) o la grilla en pantalla. `percepciones_arba()` (`services.py:3131`) incluye `_linea_txt_arba()` (`services.py:3203`), que genera además el formato de texto plano exigido por ARBA para la presentación de percepciones — un caso de exportación a formato regulatorio externo, no sólo PDF interno.

2.5 Integración AFIP — afip.py

Cliente SOAP nativo (sin `pyafipws`) para los dos servicios de Facturación Electrónica de AFIP: **WSAA** (autenticación) y **WSFEv1** (numeración y CAE).

class NumeracionYCAEProvider(ABC) — afip.py:80

Interfaz (Strategy, ver §1.3) con dos métodos abstractos:

Método	Contrato
<code>ultimo_comprobante(punto_venta: int, tipo_cbte: int) -> int</code>	Último número de comprobante autorizado por AFIP para ese punto de venta/tipo — usado para calcular el próximo número antes de emitir.
<code>solicitar_cae(...) -> ResultadoCAE</code>	Solicita el Código de Autorización Electrónica para un comprobante ya armado.

class AfipWSFEv1Stub(NumeracionYCAEProvider) — afip.py:114

Propósito	Implementación sin red: devuelve un CAE simulado. Usada en desarrollo y en la suite de tests (evita depender de credenciales/certificados reales de AFIP para poder ejecutar <code>pytest</code>).
Gatillo	<code>entorno_afip()</code> (ver abajo) resuelve a "STUB" — valor por defecto si <code>FCMENU_AFIP_ENTORNO</code> no está seteada.

class AfipWSFEv1Cliente(NumeracionYCAEProvider) — afip.py:217

Método	Ficha técnica
<code>__init__(cuit_emisor, certificado_path, clave_privada_path, homologacion=True)</code> (afip.py:251)	Configura el cliente contra Homologación (servidores de prueba) o Producción según el flag.
<code>_token_vigente() -> bool</code> (afip.py:265)	Verifica si el token de acceso (TA) obtenido de WSAA sigue dentro de su ventana de validez, para evitar reautenticar en cada llamada.
<code>_generar_tra() -> bytes</code> (afip.py:270)	Arma el XML del Ticket de Requerimiento de Acceso (TRA) exigido por WSAA.
<code>_firmar_tra(tra: bytes) -> str</code> (afip.py:288)	Propósito: firma el TRA en CMS/PKCS7 usando <code>cryptography</code> con el certificado/clave privada de la empresa. Gatillo: cada vez que <code>_token_vigente()</code> es <code>False</code> . Contrato: <code>bytes</code> (XML crudo) → <code>str</code> (CMS en base64, formato exigido por WSAA).
<code>_autenticar() -> None</code> (afip.py:310)	Orquesta <code>_generar_tra</code> → <code>_firmar_tra</code> → llamada SOAP a WSAA (<code>loginCms</code>) → guarda el Token/Sign resultante.
<code>_cliente_wsfe()</code> (afip.py:344)	Construye/cachea el cliente <code>zeep</code> para el WSDL de WSFEv1.
<code>_auth_request() -> dict</code> (afip.py:353)	Arma el bloque <code>FEAuthRequest</code> (CUIT + Token + Sign) exigido en cada llamada a WSFEv1.
<code>_errores_de(respuesta) -> str</code> (afip.py:357, staticmethod)	Extrae y concatena el detalle de errores de una respuesta SOAP de AFIP para mostrarlo al operador.
<code>_llamar(nombre, operacion, **kwargs)</code> (afip.py:364, staticmethod)	Wrapper genérico de invocación SOAP con manejo uniforme de excepción/log de la operación.
<code>ultimo_comprobante(punto_venta, tipo_cbte) -> int</code> (afip.py:376)	Implementación real contra <code>FECompUltimoAutorizado</code> de WSFEv1.
<code>solicitar_cae(...) -> ResultadoCAE</code> (afip.py:390)	Implementación real contra <code>FECAESolicitar</code> de WSFEv1; arma el <code>FeDetReq</code> con los importes netos/IVA/percepciones del comprobante.

entorno_afip() -> str / etiqueta_entorno_afip(entorno=None) -> str — afip.py:188 / afip.py:199

Propósito	Resuelve el entorno activo (<code>STUB</code> / <code>HOMOLOGACION</code> / <code>PRODUCCION</code>) desde <code>FCMENU_AFIP_ENTORNO</code> , y su etiqueta visual ("Homologación / Prueba" para <code>STUB/HOMOLOGACION</code> , " PRODUCCIÓN " sólo para el último).
Gatillo	<code>MainMenuWindow</code> la consulta para pintar el cartel de entorno permanentemente visible — control de riesgo deliberado para que el operador nunca confunda un ambiente de prueba con producción.

generar_qr_afip(...) — afip.py:485

Propósito	Genera la URL/payload del código QR obligatorio en todo comprobante electrónico argentino (RG 4892/2020 de AFIP).
Contrato de verificación	<code>tests/test_afip.py</code> decodifica el QR generado y compara los valores contra el ejemplo real que el legacy tenía hardcodeado en <code>EmiFact.frm:2785 (Obtener_QR)</code> — se documenta explícitamente como corrección de un bug de "QR estático" del legacy (el VB6 embebía siempre el mismo QR de ejemplo en vez de generarlo con los datos reales del comprobante).

2.6 Generación documental — `pdf.py` / `referencias.py`

`generar_pdf_factura(datos: DatosFacturaPDF, directorio_salida: Optional[Path] = None) -> Path` — pdf.py:92

Propósito	Renderiza el PDF de Factura Electrónica A/B con ReportLab: cabecera, detalle de renglones, totales, CAE/vencimiento y el QR AFIP (dibujado nativamente con <code>reportlab.graphics.barcode.qr</code> , sin librería externa de QR).
Gatillo	Inmediatamente después de que <code>EmissionFacturaService.emitir_factura()</code> retorna <code>ResultadoEmission</code> con éxito.
Contrato	<code>DatosFacturaPDF</code> (dataclass, pdf.py:56) → <code>Path</code> del archivo generado en <code>migration/pdf_output/</code> (o <code>directorio_salida</code> si se especifica). Nombre de archivo determinístico vía <code>_nombre_archivo()</code> (pdf.py:87, privado).
Excepciones	No captura excepciones de ReportLab; se propagan al llamador (la UI debe envolver la llamada si quiere mostrar un mensaje amigable).
Nota	<code>_dibujar_sello_borrador()</code> (pdf.py:222) stampa un sello "BORRADOR" cuando el comprobante se genera sin CAE aprobado (ej. entorno STUB) — control visual para que nunca se confunda un PDF de prueba con uno fiscalmente válido.

`generar_pdf_recibo(datos: DatosReciboPDF, directorio_salida=None) -> Path` — pdf.py:280

Mismo patrón que `generar_pdf_factura`, para el comprobante de Recibo (no lleva CAE/QR — el Recibo no es un comprobante fiscal electrónico).

`generar_pdf_listado(...)` — pdf.py:422

Motor genérico de reportes tabulares en PDF, consumido por todos los métodos de `ListadosService` (§2.4).

`generar_pdf_referencias(directorio_salida=None, *, forzar=False) -> Path` — referencias.py:89

Propósito	Genera el PDF de referencia con la matriz de códigos de sección/segmento (equivalente documental de <code>Referencias_FCMENU.pdf</code> ya publicado en <code>assets/docs/</code>).
Contrato	<code>forzar=True</code> regenera aunque ya exista un PDF vigente; por defecto reutiliza el existente (caché simple basado en existencia de archivo).

2.7 Capa de presentación (PyQt6) — patrones y componentes base

`aplicar_tema(app: QApplication) -> None` — theme.py:233

Propósito	Fachada única de estilo: aplica la hoja de estilo QSS (<code>hoja_de_estilo()</code> , theme.py:74) y la paleta corporativa (<code>class Verde</code> , theme.py:53) a toda la aplicación.
Gatillo	Primera línea de cada <code>main()</code> de entry point, antes de instanciar cualquier ventana (ver §2.9).
Efecto colateral notable	<code>_instalar_limite_combos()</code> (theme.py:40, invocado al importar el módulo) parchea <code>QComboBox.__init__</code> a nivel de clase para fijar <code>setMaxVisibleItems</code> en todos los combos de la aplicación sin tener que configurarlo ventana por ventana — técnica de <i>monkeypatching</i> deliberado y documentado, no accidental.

`icono_app() -> QIcon` / `logo_empresa(alto: int = 52) -> QPixmap` — theme.py:241 / theme.py:252

Centralizan la carga de `assets/Logo-Alestele.png`/ícono de aplicación — evitan rutas hardcodeadas repetidas en cada ventana.

Jerarquía de controles numéricos — `widgets.py:132-268`

```

QLineEdit
├── _NumericLineEditBase      # filtrado de tecla por tecla vía keyPressEvent/focusInEvent
│   ├── MontoLineEdit        # Decimal, formato es-AR (usa format_decimal/parse_decimal)
│   ├── EnteroLineEdit        # int puro
│   └── AlfabeticoLineEdit    # fuerza mayúsculas (réplica de campos "código" del VB6)

```

Clase / método	Ficha técnica
<code>MontoLineEdit.decimal() -> Decimal / set_decimal(valor: Decimal) -> None (widgets.py:223/226)</code>	Getter/setter tipado — el resto del sistema nunca lee <code>.text()</code> crudo de un campo de monto, siempre pasa por este par.
<code>_NumericLineEditBase._caracter_permitido(texto: str) -> bool</code>	Método de extensión (Template Method): cada subclase decide qué caracteres acepta tecla por tecla, antes de que el carácter llegue al widget — reemplazo directo de la validación de <code>KeyPress</code> que hacían los controles OCX legacy.
<code>EnterAsTabFilter (widgets.py:278-402)</code>	Propósito: <code>QObject</code> instalable como <code>eventFilter</code> de una ventana completa; intercepta <code>Qt.Key.Key_Return/Key_Enter</code> y los traduce a avance de foco (<code>_avanzar</code>) hacia el siguiente widget en el orden de tabulación, con soporte de <code>boton_final</code> (qué botón activar si Enter se presiona en el último campo). Gatillo: instalado una vez por ventana en su constructor. Justificación: requisito explícito de usabilidad del cliente (<code>CLAUDE.md §3</code> : "El evento Enter debe comportarse como Tab").
<code>crear_boton_hoy(campo: QDateEdit) -> QPushButton (widgets.py:82)</code>	Botón que fija un <code>QDateEdit</code> a la fecha de hoy — patrón repetido en todo formulario con filtro de fecha.

`class TablaBusqueda(QTableWidget)` — `widgets.py:500`

Método	Contrato
<code>cargar_filas(...) (widgets.py:532)</code>	Puebla la grilla a partir de una lista de objetos de dominio (<code>Cliente</code> , <code>Articulo</code> , ...), guardando el objeto original en cada fila (no sólo su representación textual).
<code>objeto_en_fila(fila: int) -> Any / objeto_seleccionado() -> Any</code>	Recupera el objeto de dominio asociado a una fila — patrón que evita relecturas a la base al seleccionar una fila en una grilla de búsqueda.
<code>_ItemOrdenable(QTableWidgetItem)</code> (<code>widgets.py:426</code>) sobrescribe <code>__lt__</code> para que el orden de columna en la grilla compare por valor numérico (<code>_valor_numerico()</code> , vía <code>Decimal</code>) cuando corresponde, no por comparación de texto — evita el bug clásico de Qt donde "10" ordena antes que "9" en una columna numérica mostrada como texto.	

`cuit_valido(cuit: str) -> bool` — `validators.py:14`

Propósito	Valida el dígito verificador de un CUIT argentino.
------------------	--

Contrato	Traducción 1:1 de <code>FCMENU.bas Function Cuit()</code> — incluye los casos borde propios del algoritmo módulo-11 (<code>verificador 11→0, 10→9</code>).
Verificación	<code>tests/test_ui_validators.py</code> lo valida contra CUITs reales conocidos (ej. el CUIT público de AFIP como organismo).

`email_valido(email: str) -> bool` — `validators.py:36`: validación de formato (regex), sin verificación de existencia del dominio/casilla.

Motor de código de artículo — `ui/articulo_codigo.py`

Compone `Articulo.COD2` a partir de segmentos definidos dinámicamente en `Fcttabla1` (cada Sección define su propia composición de segmentos — ej. la sección "GPN" usa `MM1` (espesor) + `TELAS` (cantidad de telas)). Es el motor que reemplaza la lógica de armado de código específica de cada sección que el legacy tenía hardcodeada por formulario.

Motor de renglón de factura — `ui/factura_renglon.py` (347 líneas)

Elemento	Ficha técnica
<code>resolver_seccion_renglon(fcttablas, cod_seccion) -> Optional[SeccionRenglon]</code> (<code>factura_renglon.py:131</code>)	Propósito: dado un código de sección, resuelve su configuración completa de segmentos (<code>SeccionRenglon</code> , dataclass con lista de <code>SegmentoRenglon</code>) consultando <code>Fcttabla1</code> . Excepciones: el llamador debe manejar <code>None</code> (sección inexistente) — quien sí levanta excepción tipada es la capa que envuelve esta función en la ventana (<code>SeccionInexistenteError</code>).
<code>SeccionInexistenteError / SeccionSinUnidadFacturacionError</code> (<code>factura_renglon.py:98/105</code> , subclases de <code>ValueError</code>)	Errores de dominio tipados — únicos del sistema con jerarquía propia (ver justificación en §1.3).
<code>calcular_precio_e_importe(...)</code> / <code>calcular_precio_preview(...)</code> (<code>factura_renglon.py:264/302</code>)	Motor de cálculo de precio unitario e importe de un renglón, considerando lista de precios del artículo, cotización (si aplica venta en dólares) y segmentos del código.
<code>armar_codigo_renglon(seccion, valores: list[Decimal]) -> str</code> (<code>factura_renglon.py:229</code>)	Compone el string de código final a partir de los valores ingresados por el operador en cada segmento.
<code>descripcion_con_segmentos(...)</code> (<code>factura_renglon.py:334</code>)	Arma la descripción textual del renglón incluyendo los valores de segmento (ej. "GOMA PLANCHA NAT. 3mm 1 Tela").

Cubierto exhaustivamente por `tests/test_factura_renglon.py` (297 líneas) contra datos reales de la sección "GPN" (Goma Plancha Nat.).

2.8 Catálogo de ventanas y diálogos

Toda ventana de `ui/` sigue uno de dos patrones estructurales:

- **Ventana de consulta/listado** (`*_window.py` con sufijo `_busqueda/_consulta`): construye una `TablaBusqueda`, un formulario de filtro y delega el filtrado en el `Repository/Service` correspondiente. Ejemplos: `cliente_busqueda_window.py`, `articulo_busqueda_window.py`, `cheques_consulta_window.py`, `stock_consulta_window.py`, `despachos_consulta_window.py`, `facturas_emitidas_window.py`.
- **Ventana/diálogo de edición (ABM)** (`*_detalle_dialog.py`, `*_window.py` de captura): arma un formulario con controles de `widgets.py`, instala `EnterAsTabFilter`, y en el `accept()`/botón "Grabar" arma el `dict`/dataclass de entrada y llama al `Service` correspondiente — nunca escribe SQL ni maneja la `Session` directamente. Ejemplos: `cliente_detalle_dialog.py`, `articulo_detalle_dialog.py`, `cheque_detalle_dialog.py`,

nota_cliente_dialog.py, nota_articulo_dialog.py, dtos_cliente_dialog.py, cotizacion_window.py, parametros_window.py.

La tabla siguiente mapea cada módulo de `ui/` (excluidos los `main_*.py`, cubiertos en §2.9) contra el formulario VB6 que reemplaza, según la correspondencia documentada en los docstrings de cada archivo y confirmada contra el inventario de `.frm` en la raíz del repositorio:

Módulo Python	Formulario VB6 de origen	Responsabilidad única
<code>main_menu_window.py</code>	<code>FCMENU.frm</code>	Ventana MDI principal, sidebar de navegación, barra de tareas, cartel de entorno AFIP y de conectividad
<code>facturador_window.py</code>	<code>EmiFact.frm</code> / <code>CabFact.frm</code> / <code>DetFact.frm</code>	Captura y emisión de Factura A/B con CAE
<code>recibo_window.py</code>	<code>EmiRec.frm</code> / <code>DetPago.frm</code> / <code>CabRec.frm</code>	Captura y emisión de Recibo (cobranza)
<code>ctacte_window.py</code>	<code>CtaCte.frm</code>	Consulta de extracto de cuenta corriente
<code>cliente_busqueda_window.py</code>	<code>BusClte.frm</code>	Búsqueda de clientes
<code>cliente_detalle_dialog.py</code>	<code>Abmclte.frm</code>	Alta/baja/modificación de cliente
<code>articulo_busqueda_window.py</code>	<code>Busqueda.frm</code>	Búsqueda de artículos
<code>articulo_detalle_dialog.py</code>	<code>AbmArt.frm</code>	Alta/baja/modificación de artículo
<code>tablas_window.py</code>	<code>Tablas.frm</code> / <code>Acttabla.frm</code>	ABM de tablas paramétricas genéricas (<code>Fcttabla1</code>)
<code>listados_window.py</code>	<code>Listados.frm</code>	Selector y disparador de reportes (<code>ListadosService</code>)
<code>mod_precios_window.py</code>	<code>ModPrec.frm</code>	Modificación masiva de precios por rango/sección
<code>stock_consulta_window.py</code>	<code>Stock.frm</code> / <code>Verstock.frm</code>	Consulta de stock por artículo
<code>stock_movimiento_window.py</code>	<code>Stock.frm</code> (alta)	Alta de movimiento de stock
<code>despachos_consulta_window.py</code> / <code>despacho_selector_dialog.py</code>	<code>StockDespa.frm</code> / <code>VerDesp.frm</code>	Consulta y selección de lotes/despachos con stock
<code>dtos_cliente_dialog.py</code>	<code>DtosxClte.frm</code>	Descuentos en cascada por Cliente/Sección
<code>nota_cliente_dialog.py</code>	<code>NotaClte.frm</code>	Notas libres por cliente
<code>nota_articulo_dialog.py</code>	<code>Notartic.frm</code>	Notas libres por artículo
<code>cheques_consulta_window.py</code> / <code>cheque_detalle_dialog.py</code> / <code>cheques_cliente_dialog.py</code>	<code>VerCheq.frm</code>	Cartera de cheques de terceros

Módulo Python	Formulario VB6 de origen	Responsabilidad única
<code>banco_busqueda_dialog.py</code>	(selector auxiliar, sin <code>.frm</code> de captura propio)	Selección de Banco/Sucursal para cheques
<code>cotizacion_window.py</code>	<code>Cotizac.frm</code>	Alta de cotización diaria del dólar
<code>parametros_window.py</code>	<code>Paramet.frm</code>	Edición de parámetros globales del sistema
<code>arreglo_ctacte_window.py</code>	<code>ZZGenDB.frm</code> (equivalente funcional)	Corrección manual de un movimiento de cuenta corriente
<code>arreglo_subdiario_window.py</code>	<code>Acttabla.frm</code> (equivalente funcional)	Corrección manual de un movimiento de subdiario IVA
<code>cobranzas_zona_window.py</code>	(reporte derivado de <code>CtaCte.frm</code>)	Cobranzas pendientes agrupadas por zona
<code>ventas_seccion_window.py</code> / <code>ventas_articulo_window.py</code>	<code>ESTADIST.frm</code> / <code>VTAXART.frm</code>	Estadística de ventas por sección / por artículo
<code>totales_diarios_window.py</code> / <code>total_diario_detalle_dialog.py</code> / <code>total_mensual_dialog.py</code>	<code>TotFact.frm</code>	Consulta de totales diarios/mensuales de facturación
<code>facturas_emitidas_window.py</code> / <code>factura_emitida_detalle_dialog.py</code>	<code>VerFact.frm</code>	Consulta de facturas ya emitidas
<code>saldos_clientes_dialog.py</code>	(reporte derivado de <code>CtaCte.frm</code>)	Listado de saldos de todos los clientes
<code>acerca_de_dialog.py</code>	(estándar Windows, sin <code>.frm</code> de negocio)	Información de versión/licencia
<code>procesando_dialog.py</code>	(patrón UX transversal)	Modal de progreso indeterminado durante operaciones largas
<code>pdf_preview_dialog.py</code>	(equivalente a <code>PDF.frm</code>)	Visor embebido de PDF generado antes de imprimir

2.9 Entry points y ETL

Patrón de entry point (24 archivos `main.py/main_*.py` en `ui/`, ~27 líneas cada uno, estructuralmente idénticos):

```
def main() -> int:
    app = QApplication(sys.argv)
    aplicar_tema(app)
    ventana = <VentanaEspecífica>()
    ventana.show()
    return app.exec()

if __name__ == "__main__":
    sys.exit(main())
```

Cada archivo permite levantar **una sola ventana de forma aislada** (`python -m migration.ui.main_facturador`, etc.) — es el mecanismo de prueba manual/demo del proyecto, independiente de `main_menu.py` (el entry point real de la aplicación completa, que instancia `MainMenuWindow`). Ver tabla completa de correspondencia archivo → ventana en el árbol de directorio

(§1.2); no se repite aquí por ser 1:1 con el nombre del archivo (`main_facturador.py` → `FacturadorWindow`, `main_ctacte.py` → `CtaCteWindow`, etc.).

ETL de migración de datos — `migration/etl/`

Script	Ficha técnica
<code>export_access_to_csv.ps1</code>	Propósito: vuelca cada tabla de <code>fcmenu.mdb</code> a un CSV en <code>migration/etl/data/</code> , vía DAO 32-bit (requiere PowerShell de 32 bits en Windows, por la naturaleza del driver Access legacy). Gatillo: ejecución manual, una vez por migración de datos históricos — no es parte del runtime de la aplicación.
<code>cargar_tabla(db: Session, archivo: str, modelo) -> int</code> (<code>load_csv_to_postgres.py:101</code>)	Propósito: carga un CSV a la tabla PostgreSQL correspondiente, tipando cada columna según el modelo SQLAlchemy destino. Contrato: (<code>Session</code> , <code>nombre_archivo</code> , <code>clase_modelo</code>) → cantidad de filas insertadas.
<code>_convertir_valor(raw: str, columna)</code> (<code>load_csv_to_postgres.py:83</code>)	Conversión de texto crudo del CSV al tipo Python correcto (<code>Decimal</code> para <code>Numeric</code> , <code>date</code> para <code>Date</code> , etc.) — respeta la misma regla de "nunca <code>float</code> para dinero" que el resto del sistema.
<code>main()</code> (<code>load_csv_to_postgres.py:135</code>)	Orquesta la carga de las 16 tablas con datos reales migrados (ver inventario completo en §3.1), en el orden correcto dado que no hay <code>ForeignKey</code> que fuerce un orden de inserción (ver nota en §3.2).

Capítulo 3 — Capa de Persistencia y Base de Datos (PostgreSQL)

3.1 Modelo de datos — inventario de entidades

`migration/models.py` (726 líneas) declara **21 clases** sobre `DeclarativeBase` (SQLAlchemy 2.0, sintaxis `Mapped[]/mapped_column`). Todas las tablas usan una **PK sintética `id` autoincremental** además de (o en ausencia de) la clave de negocio real del `.mdb` — decisión de fidelidad + practicidad: el `.mdb` original en varias tablas (`Ctasctes`, `Imputacion`, `Efectivo`, `MovimVS`) **no tenía ninguna PK**, así que se agrega una sintética para poder usar el patrón `BaseRepository[T].read(id)` de forma uniforme en las 21 entidades.

Hallazgo de mantenimiento: `models.py:__all__` (línea 708) exporta 16 símbolos (`Base` + 15 modelos) pero el módulo define **21 clases modelo**. `NotaArticulo`, `DtoxClte`, `Despacho`, `MovStock` y `Totales` — las cinco tablas agregadas más recientemente (2026-08-15/16, según sus docstrings) — no figuran en `__all__`. Esto no rompe nada hoy (Python permite `from .models import DtoxClte` igual, y `repository.py/services.py` ya las importan directamente por nombre), pero es inconsistente y debería corregirse en la próxima sesión de mantenimiento de este archivo (ver checklist de §4.1).

#	Clase	Tabla real	Clave de negocio	Filas reales migradas	Nota de esquema relevante
1	<code>Cliente</code>	<code>Cientes</code>	<code>CODIGO</code>	—	Padrón de clientes; <code>DEUDA</code> es el saldo cacheado (recalculado por los servicios de <code>cta.cte</code> .)
2	<code>Articulo</code>	<code>Articulo</code>	<code>COD1+COD2</code>	—	Padding fijo de Access en <code>COD1/COD2</code> (ver nota §2.3)
3	<code>Stock</code>	<code>STOCK</code>	<code>COD1+COD2</code>	—	1:1 lógico con <code>Articulo</code>
4	<code>Fctabla1</code>	<code>Fctabla1</code>	<code>CTAB+COD</code>	—	Tabla paramétrica genérica multipropósito (catálogo de secciones, condiciones de venta, provincias, zonas, etc., discriminadas por <code>CTAB</code>)
5	<code>Parametro</code>	<code>Parametro</code>	<code>CLAVE</code> (fila única '1')	—	Singleton de configuración global (IVA, punto de venta, límites)
6	<code>Ctascte</code>	<code>Ctasctes</code>	(sin PK real)	—	Movimientos de cuenta corriente — corazón contable del sistema
7	<code>Imputacion</code>	<code>Imputacion</code>	(sin PK real)	52.227	<code>TIPO/TIPOI</code> son <code>String(2)</code> , no numéricos (quirk de Access confirmado vía DAO)
8	<code>FcivaVta</code>	<code>FcivaVta</code>	—	—	Subdiario de ventas (IVA)
9	<code>Fcestad1</code>	<code>Fcestad1</code>	—	—	Línea estadística por renglón de venta
10	<code>NotaClte</code>	<code>NOTACLTE</code>	<code>CLTE</code>	—	4 juegos de campos <code>Memo</code> (<code>Text</code> , sin límite)
11	<code>NotaArticulo</code>	<code>Notartic</code>	<code>COD1+COD2</code>	—	5 juegos de campos <code>Memo</code> ; agregada 2026-08-15, no en <code>__all__</code>
12	<code>DtoxClte</code>	<code>DtoxClte</code>	<code>CLTE+SECCION</code>	12.035	<code>RG01-RG03</code> confirmados muertos en datos reales (siempre 0) — se mantienen en el modelo por fidelidad de esquema pero no se exponen en la UI
13	<code>Despacho</code>	<code>Despachos</code>	<code>COD1+COD2+NRODESP</code>	6.161	Lotes con <code>STOCK > 0</code> genuino; agregada 2026-08-16
14	<code>MovStock</code>	<code>MovStock</code>	—	244.310	Historial completo de movimientos de stock; agregada 2026-08-16

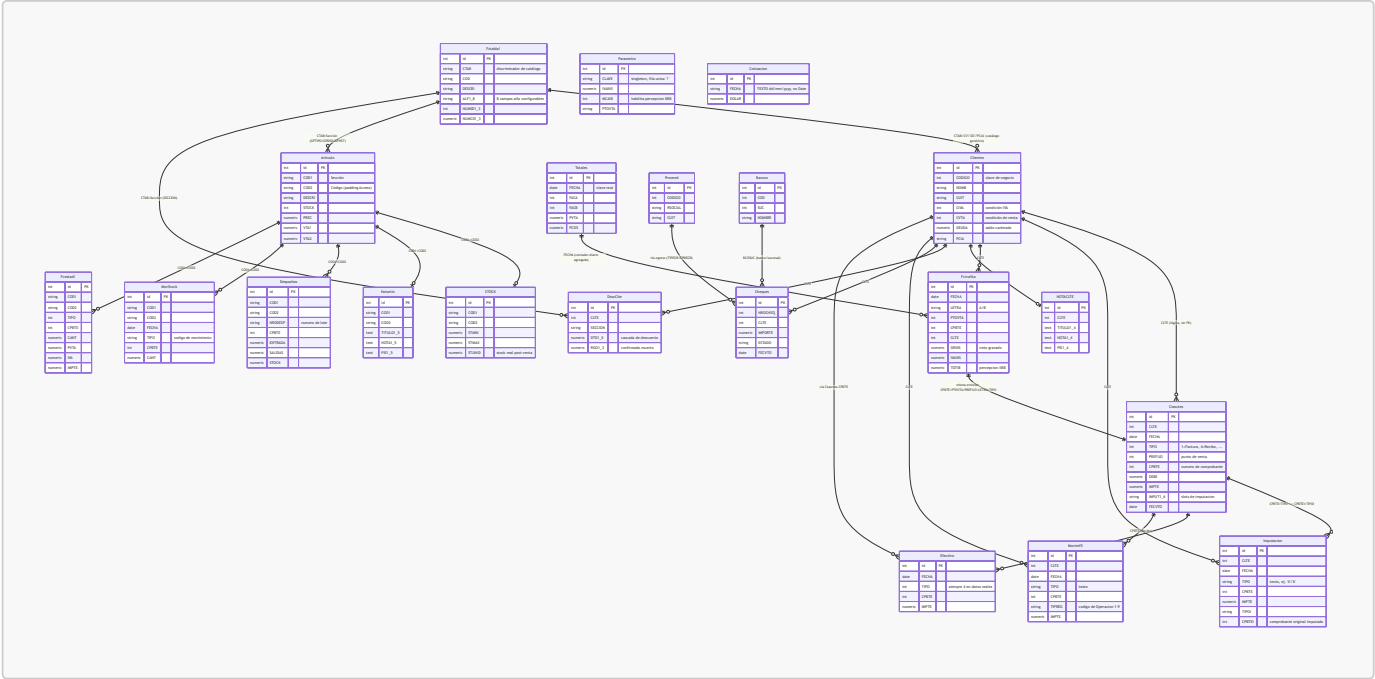
#	Clase	Tabla real	Clave de negocio	Filas reales migradas	Nota de esquema relevante
15	Totales	Totales	FECHA (real)	3.730	Contador diario agregado (~10 años de historial); sólo un subconjunto de columnas se escribe hoy (alcance Factura A/B)
16	Proveed	Proveed	CODIGO	—	Padrón de proveedores
17	Banco	Bancos	COD+SUC	—	Catálogo de bancos/sucursales
18	Cheque	Cheques	NROCHEQ	—	Cartera de cheques de terceros
19	Cotizacion	Cotizacion	FECHA (texto, no Date)	—	Único caso de fecha modelada como <code>String(10)</code> — ver nota §2.3
20	Efectivo	Efectivo	(sin PK real)	6.164	Pago en efectivo de un Recibo
21	MovimVS	MovimVS	(sin PK real)	46.929	Anticipos/Retenciones/Tarjetas/Transferencias/Baja Incobrable de un Recibo; TIPO/TIPREG son <code>String(2)</code> (mismo quirk que <code>Imputacion</code>)

Tipos de datos y convenciones aplicadas uniformemente:

- Todo importe monetario: `Numeric(12, 2)` (excepto cotizaciones de moneda, `Numeric(12, 4)`, por necesitar más precisión decimal). Nunca `Float/Double`.
- Todo campo `Memo` real de Access (`type=12`, sin longitud fija): `Text`, no `String(n)` — para no truncar contenido histórico al migrar (aplica a `NotaClte`, `NotaArticulo`, y `Proveed.REFER1`).
- Toda fecha real de Access (`dbDate`): `Date`. La única excepción es `Cotizacion.FECHA` (texto, ver arriba).
- `nullable=False` sólo en las columnas que efectivamente tienen esa restricción confirmada contra el `.mdb` real (ej. `Cliente.CODIGO`, `Parametro.CLAVE`) — el resto es `Optional[...] nullable=True` por defecto, reflejando que Access no impone `NOT NULL` salvo excepción explícita.

3.2 Diagrama Entidad-Relación (ERD)

Nota de fidelidad arquitectónica, no simplificación editorial: ninguna de las 21 tablas tiene una columna `ForeignKey()` declarada en `models.py`. Esto reproduce fielmente el `.mdb` de origen (MS-Access sin integridad referencial declarada) y es una decisión consciente del proyecto: la relación entre entidades es **lógica**, resuelta en la capa `repository.py` vía columnas de clave de negocio (`CLTE`, `COD1+COD2`, `CPBTE`, etc.), no forzada por constraint de base de datos. El ERD siguiente documenta esas relaciones lógicas — cada una está respaldada por un método concreto de `repository.py` que la implementa como filtro, listado en la columna "Evidencia".



3.3 Estrategia de conexión y Connection Pooling

migration/db.py centraliza toda la configuración de conexión en 44 líneas:

```
DATABASE_URL = os.environ.get("DATABASE_URL", "sqlite:///fcmenu.sqlite3")
_kwargspool = {} if DATABASE_URL.startswith("sqlite") else {"pool_size": 30, "max_overflow": 20}
engine = create_engine(DATABASE_URL, future=True, **_kwargspool)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)
```

Aspecto	Configuración	Justificación
Driver ORM	SQLAlchemy 2.0 (future=True en engine y sessionmaker — API "2.0-style" sin el comportamiento legacy de 1.x)	Alineado con el resto del proyecto: tipado estricto, Mapped[] en models.py.
Driver DBAPI	psycopg2 (vía postgresql+psycopg2:// en la URL) contra PostgreSQL; sqlite3 estándar contra SQLite	psycopg2-binary evita depender de headers de compilación en Windows.
Fallback de desarrollo	Si DATABASE_URL no está seteada, cae a sqlite:///fcmenu.sqlite3	Permite levantar cualquier ventana sin tener PostgreSQL instalado — no es el motor de los tests (tests/conftest.py usa su propio engine SQLite en memoria, aislado por test).
Connection Pooling	pool_size=30, max_overflow=20 (hasta 50 conexiones concurrentes) — sólo aplica a PostgreSQL; SQLite no acepta estos kwargs (usa su propio pool interno) y por eso _kwargspool es condicional	El default de SQLAlchemy (5+10=15) resultó insuficiente en un smoke test real: cada

Aspecto	Configuración	Justificación
		ventana abre su propia <code>Session</code> vía <code>get_session()</code> y no la cierra hasta que el usuario cierra la ventana. Con 15 conexiones, la ventana N°19 abierta simultáneamente se colgaba esperando una conexión libre, sin ningún error visible — el incidente que motivó subir el límite está documentado en el comentario del propio archivo.
Configuración por entorno	<code>DATABASE_URL</code> vía variable de entorno (<code>.env</code> , no versionado) — ejemplo real de desarrollo: <code>postgresql+psycopg2://fcmenu_app:***@localhost:5432/fcmenu_dev</code> , rol dedicado <code>fcmenu_app</code> (no el superusuario <code>postgres</code>)	Evita credenciales en el repositorio; separa el rol de aplicación del rol administrador de la base, buena práctica de mínimo privilegio.

Advertencia de capacidad para escalar el pool. Si el número de ventanas simultáneas soportadas crece por encima de ~50 (`pool_size + max_overflow`), o si se introduce un modo multiusuario donde varios operadores comparten el mismo proceso backend, este límite debe revisarse junto con `max_connections` del lado del servidor PostgreSQL — hoy el límite es puramente del lado del pool de aplicación, no hay evidencia en el código de que se haya dimensionado contra la configuración real del servidor.

3.4 Ciclo de vida de sesiones

El patrón de sesión es **una `Session` por ventana**, no por operación ni por request (no hay concepto de request — es una aplicación de escritorio de proceso único):

1. Cada ventana llama `get_session()` una vez, típicamente en `__init__`, y la guarda como atributo de instancia (`self.db`).
2. Todas las llamadas a `RepositoryFactory(self.db)` dentro de esa ventana reutilizan la misma `Session` — reflejando correctamente el patrón *Unit of Work* de SQLAlchemy: los cambios dentro de una misma ventana se acumulan en la misma identidad de sesión hasta el `commit()`.
3. La `Session` **no se cierra explícitamente** en la mayoría de las ventanas al cerrarse (no hay `closeEvent` que llame `self.db.close()` de forma generalizada) — se libera cuando el proceso Python recolecta el objeto ventana, o permanece abierta si la ventana se reutiliza (patrón visto en `MainMenuWindow._mostrar()`, que cachea instancias de ventana en vez de recrearlas en cada apertura).
4. `autoflush=False` significa que SQLAlchemy **no** emite `UPDATE/INSERT` automáticamente antes de cada `SELECT` dentro de la misma sesión — los servicios controlan explícitamente cuándo se sincroniza con la base (vía `commit()` o `flush()` manual), evitando escrituras parciales accidentales por una consulta de lectura intermedia.

Recomendación de mantenimiento (no implementada hoy): cualquier extensión que agregue más ventanas al `MainMenuWindow` debería evaluar cerrar la `Session` en `closeEvent()` de cada ventana no persistente (diálogos modales, en particular) — hoy mitigado sólo subiendo el tamaño del pool, no atacando la causa (sesiones que no se liberan). Ver seguimiento en la guía de modificación segura, §4.1.

3.5 Transacciones y resiliencia (ACID)

No hay una capa de transacción declarativa (no hay decorador `@transactional` ni context manager de transacción reutilizable) — cada operación multi-tabla implementa su propio bloque `try/except` con `commit()/rollback()` explícitos, siguiendo el mismo patrón en los tres servicios transaccionalmente críticos:

```
try:
    # N operaciones db.add(...) / setattr(...) sobre múltiples tablas
    self.db.commit()
except Exception:
    self.db.rollback()
    raise
```

Presente en:

- `EmissionFacturaService.emitir_factura()` (`services.py:1385-1451`) — escribe `MovStock/Stock/Despacho` (N filas), `FcivaVta` (1 fila), `Cliente.DEUDA` (update), `Ctascte` (1 fila) y `Totales` (upsert) como una única unidad atómica.
- `EmissionReciboService.emitir_recibo()` (`services.py:1706` en adelante) — escribe `Efectivo/Cheques/MovimVS` (según medios de pago) y actualiza los slots `IMPUT1..6` de `Ctascte` vía `CuentaCorrienteService.imputar_pago()`.
- `StockMovimientoService.emitir_movimiento()` (`services.py:849`) — escribe `MovStock` y actualiza `Stock/Despacho`.

Principios ACID aplicados:

Principio	Cómo se garantiza
Atomicidad	Un único <code>commit()</code> al final del bloque; cualquier excepción intermedia dispara <code>rollback()</code> antes de re-lanzar (<code>raise</code> sin argumentos preserva el traceback original) — ninguna escritura parcial queda persistida.
Consistencia	Validada en la capa de servicio antes de tocar la base (ej. <code>StockMovimientoService.validar_seccion()</code> , las guardas de <code>ChequeService.registrar_egreso()</code>) — el patrón es "fail fast" con <code>ValueError</code> antes de abrir la sección transaccional, no confiar en constraints de base de datos (que, como se documentó en §3.2, no existen a nivel <code>ForeignKey</code>).
Aislamiento	Delegado íntegramente al nivel de aislamiento por defecto de PostgreSQL (<code>READ COMMITTED</code>) — el código no eleva explícitamente el nivel de aislamiento ni usa <code>SELECT ... FOR UPDATE</code> . Riesgo identificado, no mitigado: en un escenario multiusuario real (dos operadores emitiendo factura al mismo cliente en simultáneo), no hay bloqueo pesimista sobre <code>Cliente.DEUDA</code> — la actualización es un <code>UPDATE</code> directo sobre el objeto ya cargado en la sesión, expuesto a <i>lost update</i> bajo concurrencia alta. No hay evidencia en el código de que este escenario se haya probado bajo carga concurrente; se recomienda evaluarlo antes de habilitar más de un puesto simultáneo contra la misma base.
Durabilidad	Delegada a PostgreSQL (WAL) — no hay lógica de reintento ni cola de reprocesamiento del lado de la aplicación ante una caída de la base a mitad de una transacción. Si la conexión se pierde después del <code>commit()</code> pero antes de que la UI reciba la confirmación, el comprobante queda persistido pero el operador podría no verlo confirmado — mismo riesgo que ya existía en el legacy VB6/ADODB, no introducido por la migración.

Resiliencia ante caída de la base: no hay un mecanismo explícito de reconexión automática ni *circuit breaker*. Una excepción de conexión (`psycopg2.OperationalError/sqlalchemy.exc.OperationalError`) se propaga tal cual hasta la UI, que la muestra en un `QMessageBox.critical` sin reintentar — comportamiento aceptable para una aplicación de escritorio de un solo operador por instancia, pero a documentar como límite conocido si el sistema se expone a una red menos confiable (ej. sucursal remota sobre VPN).

Capítulo 4 — Manual de Mantenimiento, Extensibilidad y DevOps

4.1 Guía de modificación segura

Protocolo para migrar un formulario VB6 nuevo (flujo ya establecido en `CLAUDE.md`, formalizado aquí con los pasos técnicos concretos):

1. **Leer el `.frm/.bas/.cls` de origen completo** — no asumir el comportamiento por el nombre del control. Anotar: tablas de `fcmenu.mdb` tocadas, eventos de UI relevantes, controles OCX usados (para elegir su reemplazo en `widgets.py`), llamadas a AFIP si las hay.
2. **Auditar el esquema real contra `fcmenu.mdb` vía DAO 32-bit** (no confiar en el tipo de dato "aparente" del VB6) — cada modelo existente en `models.py` documenta en su docstring cómo fue auditado; seguir el mismo estándar (tipo real, longitud real, nulabilidad real, y si tiene o no PK real). Si la tabla ya existe en `models.py`, verificar que ninguna columna nueva del formulario a migrar falte en el modelo.
3. **Si la tabla no existe en el modelo:** agregar la clase en `models.py`, con:
 - Docstring documentando el origen de la auditoría (igual criterio que las 21 clases existentes).
 - Tipos siguiendo las convenciones de §3.1 (`Numeric(12,2)` para dinero, `Text` para Memo, `Date` para fecha real, `String(n)` con el largo real de Access para todo lo demás).
 - **Actualizar `__all__`** (recordar el hallazgo de §3.1 — no repetir el mismo olvido con la próxima tabla nueva).
 - Generar la revisión de Alembic correspondiente (ver §4.2), nunca editar una tabla existente a mano en la base sin pasar por una migración versionada.
4. **Agregar el repositorio en `repository.py`:** heredar de `BaseRepository[NuevoModelo]`, agregar sólo los métodos de consulta que el formulario realmente necesita (no exponer CRUD genérico si el dominio no lo requiere — ver que ninguna clase de repositorio expone `delete()` salvo que el `.frm` original tenga baja física). Registrar el nuevo repositorio en `RepositoryFactory`.
5. **Agregar el servicio en `services.py`** si hay lógica de negocio (cálculo, validación, orquestación multi-tabla) — **cada fórmula debe citar la línea exacta del `.frm` de origen en el docstring**, siguiendo el estándar ya usado en todo el archivo. Si una regla de negocio no está clara en el VB6 (parche contradictorio, SQL ineficiente, variable no usada), **detenerse y preguntar al usuario antes de asumir** — regla de oro explícita del proyecto, no opcional.
6. **Construir la ventana Python** replicando el layout original, usando los controles de `widgets.py` (nunca un `QLineEdit` plano para un campo de monto — siempre `MontoLineEdit`) e instalando `EnterAsTabFilter`. Si la ventana necesita un entry point standalone de prueba, agregar un `main_<nombre>.py` siguiendo el patrón de §2.9.
7. **Escribir los tests primero para la capa de servicio** (no para la UI — la suite del proyecto no testea Qt, testea `services.py/repository.py` contra SQLite en memoria vía la fixture `db` de `tests/conftest.py`). Cualquier bug de fórmula encontrado durante el desarrollo debe quedar como test de regresión permanente (mismo criterio que documentan los docstrings de `test_repository.py`).
8. **Ejecutar la suite completa antes de integrar** (ver comando en §4.4) — el proyecto no tiene una regla de "sólo correr los tests afectados"; dado que varios servicios comparten repositorios (ej. `CtascteRepository` es usado por seis servicios distintos), un cambio ahí exige correr todo `test_services.py`.

Protocolo para alterar una tabla existente sin romper el sistema:

- **Nunca** modificar una columna existente sin verificar primero, con `grep`, todos los lugares de `repository.py/services.py/ui/` que la leen o escriben — al no haber `ForeignKey`/constraints, el compilador de Python no va a avisar de una columna renombrada; sólo lo hará en tiempo de ejecución (o un test, si existe cobertura).
- Generar la migración con `alembic revision --autogenerate -m "<descripción>"` y **revisar el diff generado a mano** antes de aplicarlo — el `autogenerate` de Alembic no detecta cambios de tipo de columna de forma confiable en todos los dialectos, y este proyecto tiene varios casos deliberadamente "no convencionales" (`Cotizacion.FECHA` como texto, `Imputacion.TIPO` como texto) que un `autogenerate` ingenuo podría "corregir" incorrectamente.
- Si la tabla tiene datos históricos reales ya migrados (ver conteos de filas en §3.1 — varias superan las 200.000 filas), probar la migración primero contra una copia de `fcmenu_dev`, nunca directo contra producción.
- Actualizar el CSV/ETL correspondiente en `migration/etl/data/` y `load_csv_to_postgres.py` sólo si la migración de datos históricos aún no se hizo para esa tabla — si ya se migró, el ETL es código de un solo uso y no necesita mantenerse sincronizado indefinidamente (documentar en el propio script si se decide congelarlo).

Checklist de deuda técnica conocida a resolver oportunamente (detectada durante el análisis de este documento, no bloqueante):

- `models.py.__all__` desincronizado (faltan 5 símbolos) — ver §3.1.
- Ausencia de `ForeignKey` declarativas — decisión consciente de fidelidad al `.mdb`, pero vale evaluar agregarlas como constraints *no destructivas* en una migración futura, una vez que los datos históricos estén confirmados 100% consistentes (evitaría bugs de integridad silenciosos hacia adelante, sin invalidar el historial ya migrado).
- Sesiones de UI sin cierre explícito (§3.4) — mitigado por tamaño de pool, no resuelto de raíz.
- Ausencia de framework de logging (ver §4.3) — toda la trazabilidad depende hoy de mensajes en pantalla (`QMessageBox`) y de la disciplina de tests, sin rastro persistente en disco de errores en producción.

4.2 Gestión de esquema — Alembic

`alembic.ini` (raíz) + `migration/alembic/env.py` gestionan el versionado de esquema:

- `target_metadata = Base.metadata` (`env.py:22`) — habilita `alembic revision --autogenerate` contra las 21 clases de `models.py`.
- La URL de conexión se resuelve en tiempo de ejecución desde `DATABASE_URL` (mismo criterio que `db.py`), **pisando** lo que diga `sqlalchemy.url` en `alembic.ini` — evita mantener la cadena de conexión en dos lugares y commitear credenciales por accidente.
- Modo online usa `NullPool` (`env.py:71`) explícitamente — correcto para un proceso de migración de corta vida (no tiene sentido pooling para un script que corre una vez y termina).

Historial de revisiones (`migration/alembic/versions/`, 8 archivos, orden cronológico por `down_revision`):

Revisión	Descripción	Qué agrega
476bdfaf3277	Esquema inicial	14 tablas base, fiel al <code>.mdb</code> auditado (primera fase de la migración de datos)
c5d3a39ce576	Agrega <code>Efectivo</code> y <code>MovimVS</code>	Tablas de medios de pago del Recibo
1c90fa2bc07d	Agrega <code>MovStock</code> y <code>Totales</code>	Historial de stock y contador diario
b88d80214646	Agrega <code>Despachos</code>	Lotes con stock
aa584d0b6666	Agrega <code>Notartic</code>	Notas de artículo
56c4222f3b26	Agrega <code>DtoxClte</code>	Descuentos por cliente/sección
8cd556285914	Corrige tipos de <code>TIPO/TIPREG</code> de <code>MovimVS</code>	Corrección de tipo (int→string) tras confirmar el quirk de Access contra datos reales

Comandos operativos:

```
# Aplicar todas las migraciones pendientes contra DATABASE_URL
alembic upgrade head

# Generar una nueva revisión a partir de cambios en models.py (SIEMPRE revisar el diff a mano)
alembic revision --autogenerate -m "descripcion_del_cambio"

# Revertir la última revisión (uso excepcional, nunca contra datos de producción sin backup)
alembic downgrade -1

# Ver el historial de revisiones aplicadas
alembic history
```

Advertencia operativa: `db.init_db()` (`db.py:38`, `Base.metadata.create_all`) es una vía **alternativa e independiente** de Alembic para crear el esquema — útil para bootstrap rápido de un SQLite de desarrollo, pero **no debe**

usarse contra una base PostgreSQL ya gestionada por Alembic: `create_all()` no registra nada en la tabla `alembic_version`, así que mezclar ambos mecanismos deja el control de versión de esquema en un estado inconsistente (Alembic no sabría desde qué revisión partir).

4.3 Trazabilidad, logging y manejo de errores

Hallazgo verificado: una búsqueda exhaustiva de `import logging/getLogger/logging.basicConfig` en todo `migration/` **no arroja resultados** — el proyecto **no tiene un framework de logging estructurado** a la fecha de este documento. La trazabilidad de errores hoy depende enteramente de dos mecanismos:

1. **Excepciones tipadas con mensaje descriptivo en español**, capturadas en el punto de la UI más cercano al usuario y mostradas vía `QMessageBox.critical/.warning/.information` (76 invocaciones detectadas en 22 archivos de `ui/`, concentradas en las ventanas transaccionales: `facturador_window.py` (12), `tablas_window.py` (8), `recibo_window.py` (6), `articulo_detalle_dialog.py` (7), `cliente_detalle_dialog.py` (7)).
2. **La suite de tests como red de regresión** — cada bug real encontrado durante el desarrollo (ver ejemplos citados en §2.3: `deuda_cliente()`, `StockRepository.criticos()`, `ParametroRepository.get_ivains()`) quedó como test permanente en vez de como entrada de log; la "memoria" de errores pasados vive en `tests/`, no en un archivo de log rotado.

Manejo de excepciones por capa:

Capa	Estrategia
Repositorio (<code>repository.py</code>)	No captura excepciones de SQLAlchemy/DBAPI — las deja propagar. Es responsabilidad de la capa de servicio decidir si son recuperables.
Servicio (<code>services.py</code>)	Levanta <code>ValueError</code> con mensaje en español ante violación de regla de negocio (guardas explícitas, "fail fast" antes de tocar la base). En las transacciones multi-tabla, envuelve las escrituras en <code>try/except Exception: rollback(); raise</code> (ver §3.5) — nunca silencia la excepción, siempre la re-lanza tras revertir.
Integración AFIP (<code>afip.py</code>)	<code>_errores_de(respuesta)</code> centraliza la extracción de mensajes de error de la respuesta SOAP de AFIP en un único string legible; <code>ResultadoCAE.aprobado</code> (property, <code>afip.py:75</code>) permite a la UI verificar éxito sin parsear la respuesta cruda.
UI (<code>ui/*.py</code>)	Captura la excepción en el handler del botón/acción y la traduce a <code>QMessageBox.critical(self, "Título", str(excepcion))</code> — el mensaje que llega al operador es literalmente el mensaje de la <code>ValueError</code> levantada en <code>services.py</code> , por eso esos mensajes están redactados en español claro orientado al usuario final, no en jerga técnica.

Recomendación concreta para una futura iteración (no implementada hoy): introducir `logging` estándar de Python con al menos:

- Un logger por módulo (`logging.getLogger(__name__)`) en `services.py` y `afip.py` como mínimo, con `RotatingFileHandler` apuntando a un archivo local (ej. `%LOCALAPPDATA%/fcmenu/app.log` en Windows) — para poder diagnosticar remotamente un error reportado por un operador sin depender de que haya podido leer y transcribir correctamente el texto del `QMessageBox`.
- Nivel `WARNING` para reglas de negocio rechazadas (las `ValueError` ya existentes), nivel `ERROR` con `exc_info=True` para excepciones no anticipadas (fallas de conexión, errores de AFIP no mapeados), nivel `INFO` para operaciones transaccionales exitosas (emisión de factura/recibo con su número de comprobante) — esto último daría, por primera vez, un rastro de auditoría independiente de la propia base de datos.
- **No** loguear nunca el contenido de `certificado_path/clave_privada_path` ni la cadena de conexión completa (`DATABASE_URL` contiene la contraseña) — si se implementa logging de configuración, enmascarar credenciales explícitamente.

4.4 Framework de pruebas

Framework: `pytest` ≥ 8.0 (`requirements-dev.txt`), sin plugins adicionales declarados (no hay `pytest-cov`, `pytest-mock`, etc. en las dependencias — `unittest.mock.patch` se usa directamente donde hace falta, ver `test_services.py`).

Estrategia de aislamiento: cada test que necesita base de datos recibe la fixture `db` (`tests/conftest.py:19`) — una `Session` sobre un engine **SQLite en memoria nuevo por test** (`create_engine("sqlite:///memory:")`), con `Base.metadata.create_all(engine)` ejecutado antes de cada test). Esto garantiza:

- Velocidad: sin I/O de disco ni de red, la suite completa corre en segundos.
- Independencia total entre tests (ningún test puede filtrar estado a otro).
- **No** depende de PostgreSQL ni de `fcmenu.mdb` — los datos reales del `.mdb` se usaron *durante el desarrollo* para confirmar el esquema y las fórmulas (documentado como referencia externa a `data_layer_progress.md`, un artefacto de la sesión de desarrollo, no parte del repositorio de pruebas).

Inventario de la suite (`tests/`, 10 archivos + fixtures):

Archivo	Líneas	Qué cubre
<code>test_services.py</code>	2.545	La suite más grande: toda la lógica de negocio de <code>services.py</code> — IVA, cascada de descuentos, imputación de pagos, emisión de factura/recibo, movimientos de stock, estadísticas de venta. Usa <code>unittest.mock.patch</code> para aislar dependencias externas (ej. fecha "hoy" fija para tests de vencimiento).
<code>test_repository.py</code>	693	Regresión de los bugs reales de la capa Repository ya documentados en §2.3 — cada test cita el bug concreto que corrigió.
<code>test_pdf.py</code>	203	Verifica contenido real del PDF de factura (CAE, vencimiento, total) leyendo de vuelta el archivo generado con <code>pypdf.PdfReader</code> — no sólo que el archivo se haya creado.
<code>test_afip.py</code>	163	QR AFIP validado contra el ejemplo real hardcodeado del legacy (ver §2.5); generación y firma de TRA.
<code>test_articulo_codigo.py</code>	163	Motor de composición de código de artículo, contra datos reales de la sección "GPN" confirmados en <code>fcmenu_dev</code> .
<code>test_factura_renglon.py</code>	297	Motor de renglón de factura, reusa los mismos datos reales de <code>test_articulo_codigo.py</code> extendidos con Unidad de Facturación.
<code>test_decimals.py</code>	81	<code>parse_decimal()/format_decimal()</code> , incluyendo el caso del bug de formato US/es-AR documentado en §2.1.
<code>test_fechas.py</code>	22	Formato de fecha en español.
<code>test_provincias.py</code>	23	Catálogo de provincias.
<code>test_ui_validators.py</code>	42	<code>cuit_valido()</code> contra CUITs reales conocidos y casos borde del algoritmo módulo-11.

Comandos operativos:

```
# Suite completa (ejecutar desde c:\fcmenu, con el entorno virtual activado)
pytest

# Un módulo puntual, con detalle de aserciones
pytest tests/test_services.py -v

# Un caso puntual por nombre
pytest tests/test_repository.py -k deuda_cliente -v
```

Criterio de "regresión cero" del proyecto: todo bug de fórmula o de consulta encontrado durante el desarrollo se corrige **y** se deja como test permanente que reproduce el escenario exacto que falló (no un test genérico) — es la política implícita ya aplicada consistentemente en `test_repository.py` y `test_services.py`, y debe mantenerse igual en cualquier extensión futura del sistema: un bug corregido sin test de regresión asociado se considera trabajo incompleto bajo el estándar de este proyecto.

Fin del documento. Generado por auditoría directa del código fuente de `c:\fcmenu\migration` — toda referencia a línea de archivo, conteo de filas o cita de fórmula fue verificada contra el repositorio real a la fecha indicada en la portada, no asumida.